The kernel and its modules

Drishti · docs/architecture/KERNEL_AND_MODULES.md · Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

Status: design, not built. Written for 2.9.11.i, now split into **i1, i2 and i3** (section 12.2), on the product owner's request of 2026-11-08: Drishti becomes a domain-free **kernel** for BI and aggregation; capabilities arrive as separately built, signed **modules** loaded from folders; connectors stay the kernel's; a **distribution file** white-labels the product. Banking is the first module and the worked example throughout. Surveyed on `wip/train` `89f010df5`. A printable copy with the diagrams and mock-ups: KERNEL_AND_MODULES.pdf.

This is the one home for the kernel, the module package, loading, data binding, distributions and the SDK. It builds on what is already built and does not repeat it: the access chain (licence → module → lens → function → permission) and the registry are ACCESS_MODEL.md; the boundary survey, the dependency rules and the split as built are DOMAIN_MODULES.md (section 14 says what moves from it into this document); the JavaScript libraries are ELEMENTS.md; the lakes and connectors are DATA_ARCHITECTURE.md; the lenses are LENSES.md.

Worked examples (files, not product code) are in `docs/architecture/kernel-and-modules/examples/` (the banking manifest: module.yaml; the genomics sample module as a Maven project with its own pages: genomics/ (docs/architecture/kernel-and-modules/examples/genomics/project-tree.txt)); the screens' mock-ups in `docs/architecture/kernel-and-modules/mockups/` (section 15). The developer path is section 8, a module's own pages section 9, and what happens when a module is removed section 10.

Contents

1. Goals, non-goals and principles
2. The kernel
3. The module package (`.dmod`)
4. Loading
5. Data needs and connector binding
6. The distribution file (white label)
7. Lifecycle and tooling
8. The Module SDK and the developer path
9. Module UI and the developer's own pages
10. Offloading a module: everything disappears
11. Security
12. Migration, with banking as the proving ground
13. Open questions for the owner
14. What moves from DOMAIN_MODULES.md
15. Mock-ups

---

1. Goals, non-goals and principles

1.1 Goals

#	Goal	Proof
G1	**A domain-free kernel**: general BI and aggregation (packs runtime, Sutras, Rachana, Pivot, BI, tables and models, lakes, connectors, search, Explain, approvals, security, cluster, Elements core) that serves a bank, a hospital or a genome lab unchanged	the kernel boots and serves BI, Pivot and its own reference data with **no module installed**; `tools/domain_strings_allowed.yaml` is empty
G2	**Modules built and shipped separately**, like Linux kernel modules: one signed file (`.dmod`) per module, compiled against the kernel API only, dropped into a configured folder	the banking module is built in its own build against the SDK jars and passes its suite against the kernel alone
G3	**A module carries everything its capability needs**: Java compute, services and API; Python compute; packs (kinds, contracts, Sutras, samples); console routes, templates, the domain Elements library, CSS on the theme tokens; lenses, functions, Home cards; feeds; Help, guides, tutorials, glossary; settings schema; licence feature	removing `banking.dmod` leaves no banking word, page, file, Help page, engine or API anywhere (ACCESS_MODEL.md section 6, extended to "not installed")
G4	**The kernel controls connectors.** A module declares the *data* it needs (kinds and contracts); the site binds its configured connectors and lake tables to them	no connector name, plugin id, host or credential in any module file (a packager check); Admin → Data bindings shows every need and what serves it
G5	**White label by configuration**: one distribution file picks modules, brand, theme, lenses, Home and docs	`drishti.finance` and a sample `drishti.genomics` boot from configuration only, from the same kernel build
G6	**VaR exactly equal** through every phase	the 2.9.11.c VaR check, bit for bit, at the 509 and 250 windows

1.2 Non-goals

- No public module marketplace or online repository in 2.9.11.i1–i3 (a site's module repository is phase P5).
- No third-party connector plugins inside modules, ever (G4). Connector plugins remain kernel extensions (section 2.4).
- No OSGi or JPMS layer: one class loader per module with an explicit export list is enough and keeps Spring and the build simple.
- No second way: banking does not keep an "in the jar" path once it is a `.dmod` (no dual machinery: the development fat jar is a build of the kernel plus modules, not a different loading path; section 4.10).
- No hot load of code in P2 (packs and Sutras are already hot; code modules restart). Hot code load is P5. Pages, words, menus and every other surface **disappear hot** in P2 (section 10).

1.3 Principles

1. **The kernel knows no domain.** No banking word, engine id, kind, path or package in kernel code, templates, scripts or styles. The rule-4 check of DOMAIN_MODULES.md section 4 becomes a gate with an empty allow list.
2. **One way. Banking ships as a module like any other.** No privileged module, no "built-in" path. Economy and capital markets are content-only modules packaged the same way. The kernel's own reference data (calendar, GLEIF) stays kernel content.

3. **Modules reach the kernel only through the kernel API** (section 2.2): versioned SPIs and services, the Rachana grammar, the data-contract format, the Elements core API, the REST namespace given to the module, the event bus. Everything else of the kernel is invisible to a module's class loader.
4. **Modules never depend on each other's internals.** `requires:` names a module and a version range; a module may use another's *published kinds and functions* (data), never its classes.
5. **Standard libraries, standard formats.** The package is a zip with a JAR-style signature; YAML manifests; JSON Schema for settings; ES modules and `.d.ts` for JavaScript; semver for versions; data in JSONL and Delta/Iceberg via their official libraries.
6. **Secure by default.** Unsigned or untrusted modules are refused (a development key exists for the SDK harness only); licences decide what loads; Python runs in the kernel's sandbox; module assets live under a strict CSP; **secrets never enter a module**.
7. **Elements, never bespoke UI.** A module's screens are Sutras first, then composed pages, then its own domain library (drishti-risk for banking; section 9), all built from Drishti Elements on the kernel's theme tokens (ELEMENTS.md).
8. **Declared, so removable.** Everything a module shows or offers (routes, words, menus, cards, Help, jobs, SPIs) is declared and registered by the kernel, never by module code at import; removing a module withdraws the declarations, so it is absent everywhere (section 10).
9. **Refused means absent, with the reason.** A module that fails a check is not loaded at all; Health and Admin → Modules say why in words.

1.4 The Linux analogy (the owner's frame)

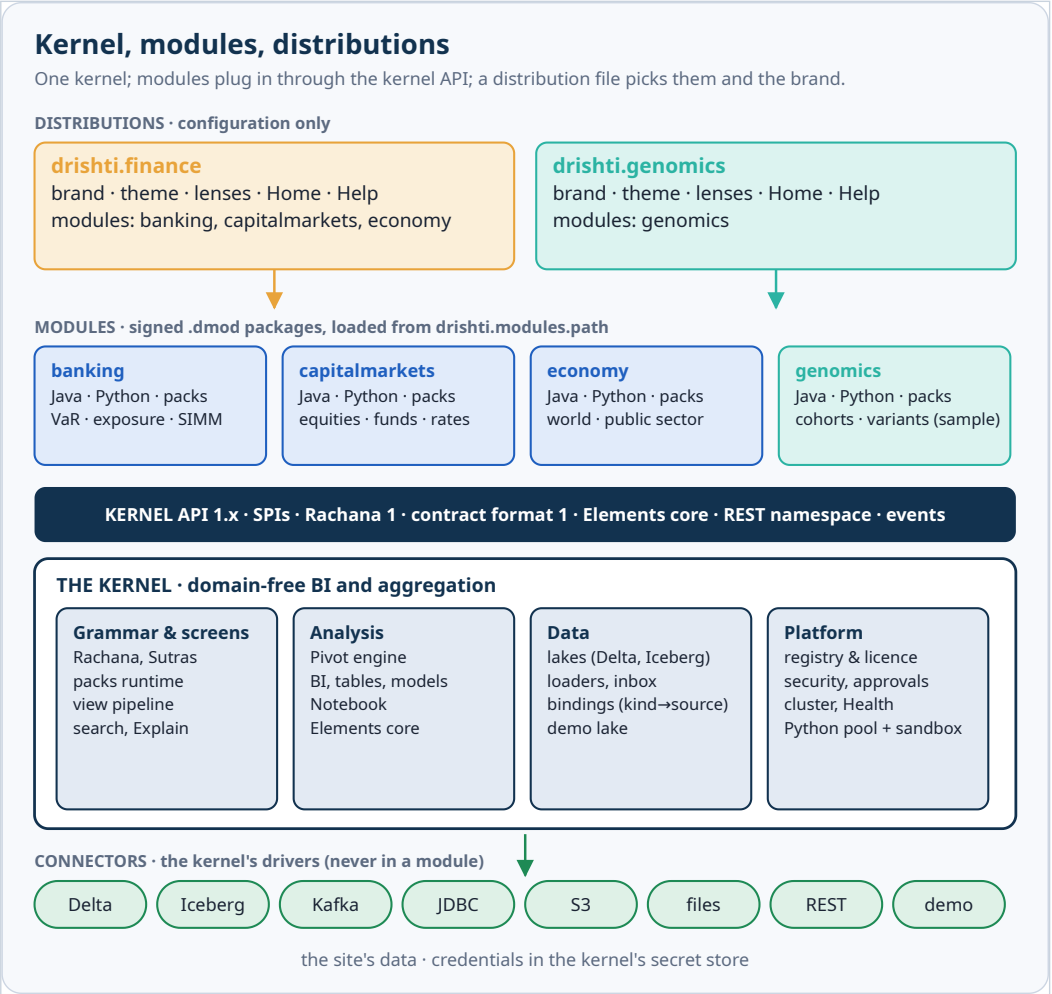
The Linux-module analogy

The owner's frame: modules are compiled and packaged apart, and the kernel runs them.

Linux	Drishti
`kernel`	Drishti kernel (server, console, Elements core, connectors)
`.ko module`	.dmod module package
`module ABI · vermagic`	kernel API version · manifest's kernel: range
`modinfo`	module.yaml · module info
`depmod · modprobe`	requires: with version ranges, resolved first
`module signing · Secure Boot`	signed .dmod · trusted publishers · licence
`insmod · rmmod · lsmod`	module install · remove · list · Admin → Modules
`taint`	Health: TAINTED (dev-signed, override …)
`device drivers`	connectors: the kernel's, bound to kinds
`distribution (Debian …)`	distribution file (drishti.finance …)

Linux	Drishti
kernel	the Drishti kernel (server + console core + Elements core + connector plugins)
`.ko` module	`.dmod` module package
module ABI, `vermagic`	the versioned kernel API and the manifest's `kernel:` range
`modinfo`	`module.yaml` (and `drishti.py module info`)
`depmod` / `modprobe` dependency resolution	`requires:` with version ranges, resolved before loading
module signing, Secure Boot	signed `.dmod`, trusted publisher keys, licence through the registry
`insmod` / `rmmod` / `lsmod`	`drishti.py module install | remove | list`, Admin → Modules
taint	Health "unsupported module loaded" (development-signed, out-of-range, or overridden)
device drivers talk to hardware via the kernel's bus	modules read data via kernel **bindings**; connectors are the kernel's drivers
distribution (Debian, Fedora)	distribution file (`drishti.finance`, `drishti.genomics`)

2. The kernel



2.1 What is in it

Area	Kernel (Maven modules / folders)
Foundations	`drishti-common`, `drishti-api` (data SPIs), `drishti-registry` (access registry, licence), `drishti-identity`
Grammar and screens	`drishti-rachana` (Rachana, Sutras), `drishti-engine` (view pipeline, inference, search, phrase, view augmenters), `drishti-packs` (pack loader and pack runtime)
Analysis	`drishti-bicompute` (pivot engine, Rachana functions, Explain, what-if), `drishti-tables` (virtual tables, models), `drishti-graph`, `drishti-inference`
Data	`drishti-lakeio`, `drishti-lakeconfig`, `drishti-lakeload`, `drishti-deltalake`, `drishti-iceberg`, `drishti-memlake`, `drishti-diskcache`, `drishti-dataload` (the inbox), `drishti-flight`, `drishti-messaging`
Connectors	every `plugins/drishti-plugin-*` (delta, iceberg, iceberg-external, file, s3, jdbc, duckdb, kafka, rabbitmq, activemq, redis, mongodb, cassandra, aerospike, rest, feeds, news, demo, yfinance); the connector files, credentials and **bindings** (section 5)
Compute	`drishti-compute` (generic: vector store, many-set sums, sharding, the Python pool and sandbox, the scheduler), `drishti-native` (BLAS/LAPACK with Java fallbacks)
Platform	`drishti-server` (REST, security, approvals, collaboration, alerts, cluster `drishti-net`, Health, audit, Admin), `drishti-ai` (LLM providers, MCP), `drishti-app` (the runnable kernel jar)
Console	`drishti-console` core: shell, command line, Home, Pivot, BI, Build, Notebook, Admin, Help centre, search, embed, phone; `core/plugins.py` hooks
JavaScript	Elements libraries 1–7 and the generic additions: drishti-core, -inputs, -grid, -chart (+ maps), -data, -insight, -renderer, -graph, -tree, -designer
Content	the core pack (calendar and generic entities), GLEIF reference data, the compute-demo pack, the kernel's Help (generic guides and tutorials), the CIO and oversight lenses, the default theme

Not in it: `drishti-banking`, `drishti-compute-banking`, `drishti-console/plugins/{banking,economy}`, the drishti-risk library, the packs of `config/packs/{banking,economy,capitalmarkets}`, their lenses, lakes, BI content, feeds, guides and tutorials. Each of those is a module (section 12.1 lists today's leaks).

2.2 The kernel API

The kernel API is a **published, versioned contract**: what a module may compile against and rely on. It is one version number for the whole surface, `kernelApi: MAJOR.MINOR`, separate from the product version (2.9.11.i2 ships kernel API **1.0**).

Surface	Form	Where
Java SPIs and services	`drishti-kernel-api-<v>.jar` (interfaces, records, annotations only; no implementation)	section 2.3
Python compute API	`drishti_kernel` package (the compute SDK's item, frame and result types; the sandbox's rules)	section 4.6
Rachana and Sutras	grammar version `rachana: 1` (already in every Sutra)	RACHANA_REFERENCE.md
Data contracts	contract format `contract-format: 1` (the `*.contract.yaml` shape the packs use today)	section 5.2
Elements core	`@drishti/elements-core` etc.: ES modules + `.d.ts`, the theme tokens	ELEMENTS.md
Console plugin API	`plugin.yaml` format 1 and the hooks of `core/plugins.py` (command, retired, type_ahead, hints, preset, landing_rule, screens)	section 4.7
REST	the namespace a module is given (`/api/v1/<module api>/…`) and the kernel's public endpoints	section 2.5
Events	the kernel event catalogue	section 2.6
Manifest	`module.yaml` format 1	section 3.3

Versioning rules.

- **Semver on the kernel API**, not on the product: a MINOR adds (a new SPI, a new optional method with a default, a new event); a MAJOR removes or changes. The product version and the kernel API version are both printed on About, Health and `drishti.py module list`.
- **Deprecation over one release** (the owner's rule): a deprecated element keeps working for one product release, logged once per start and listed on Admin → Modules against each module that still uses it ("banking 3.0.0 uses `HealthFigures.contribute(Map)`, deprecated, gone in the next release"); the next release removes it and bumps MAJOR.
- **Compatibility range**: the kernel declares `kernelApi: 1.4` and `serves: [1.3, 1.4]` (the current minor and the one before it within the major). A module declares `kernel: ">=1.3 <2.0"`. A module is loadable when its range contains the kernel's `kernelApi` and its lowest bound is not older than `serves`' lowest (the `vermagic` check, section 4.3).
- **The SDK jar is the contract**: anything not in `drishti-kernel-api` is not API, even if public in `drishti-server`. An architecture test fails a module class that imports anything else of `com.ash.drishti.*` (section 4.4).
- **API diffs are generated** each release (`japicmp` on the API jar, a `.d.ts` diff, a manifest-schema diff) and a MAJOR change without a MAJOR bump fails the gate.

2.3 The SPIs

Existing SPIs (built for the domain split; the banking module fills them today). Each moves into `drishti-kernel-api` under `com.ash.drishti.kernel.api.*`, keeping its shape; banking's use is in brackets.

SPI (today's home)	What a module contributes	Banking
`ComputeEngine` (`drishti-api` compute)	a compute engine with its switch, words and health	VaR, exposure, FRTB-SA, SIMM, sensitivities, MR cube
`PivotProvider` (`drishti-bicompute` spi)	measures the pivot engine computes per cell (non-additive totals)	VaR, ES, PVaR by any grouping
`PivotSourceProvider` (`server.pivot`)	a source the pivot asks before the tabular provider	the VaR source of the MR lens
`FunctionProvider` (`drishti-rachana` el)	Rachana functions (`var()`, `pvar()`, `es()`)	VaR functions in BI and Sutras
`ViewAugmenter` (`drishti-engine`)	fields added to a kind's view as the page opens	`VAR_ENGINE.*` on a position, trade, book
`Explainer` (`server.explain`)	the Explain drawer's formula, inputs and lineage for its numbers	VaR, component VaR, EPE, FRTB charge
`CacheGroup` / `CacheReport.Owner` (`server.caches`)	caches the kernel reports and clears in order	VaR and exposure date caches
`ApprovalSource` (`server.approvals`)	items in the Approvals inbox	regulatory VaR sign-off, scope changes
`ShardedMeasure` (`server.shard`)	a measure computed across shards	VaR by hierarchy at scale
`FlightForm` (`server.flight`)	an Arrow Flight descriptor it serves	P&L vectors out
`CapacityFigures` (`server.capacity`)	its rows in the capacity planner	VaR memory and time per date
`HealthFigures` (`server.api`)	figures in Health	hierarchies skipped
`OfficialReaders.OfficialResults` (`drishti-packs`)	published results lakes it reads	`var-official`, `ccr-official`
`ApiAlias` (`server.modules`)	old paths answered as new for one release	`/api/v1/var` → `/api/v1/risk/var`
`SourcePlugin` (`drishti-api`)	**kernel only from P1** (section 5.7): banking's `VarPositionsPlugin` and `VarPvStreamPlugin` become `KindProvider`s	—
`StorageProvider` (`drishti-lakeio`), `ProviderFactory` (`drishti-ai`)	**kernel only** (storage back-ends, LLM providers)	—

New SPIs (P1 unless noted):

SPI	Purpose	Banking
`DrishtiModule`	the module's entry point: `start(ModuleContext)`, `stop()`, `health()`; the context gives the kernel services below, its settings, its folders and its logger	`BankingModule`
`ModuleContext` services	`Kinds` (read and subscribe to bound kinds **as the person**), `KindWriter` (write produced kinds to their bound target), `Events`, `Scheduler`, `ComputePools` (incl. the Python pool), `Settings`, `Secrets` (**none**: modules get no secrets; section 11.4), `Audit`, `Registry` (ask, never read), `Caches`, `Clock` (business date)	VaR reads `scenario-pv`, `pnl-vector`, `risk-hierarchy` through `Kinds`
`KindProvider`	answers a kind the module **computes** (replaces modules' `SourcePlugin`s and the reactive-connector Java runners): fetch, search, subscribe; inputs read through `Kinds`	`position` (derived from trades), `var-result` (`VRES`), `exposure-result`, intraday scenario PVs

SPI	Purpose	Banking
`ModuleSettings`	binds the module's `settings/schema.json` to a typed record; validated at start and on change	VaR windows, confidence, root names, hierarchies
`AiToolProvider`	tools the AI assistant may call, each tied to a function (off function = no tool)	"VaR of a scope", "why VaR moved"
`RestModule` (annotation `@ModuleApi`)	marks the module's controllers; the kernel mounts them under the module's namespace (section 4.5)	`/api/v1/risk/**`
`ExportProvider` (P3)	named exports (workbook, regulatory files)	regulatory VaR files
`PlaygroundTemplates` (P3)	the generic workbook streamer's templates	VaR, FRTB-SA, SIMM, SA-CCR, FVA playgrounds
`ModuleMigration` (P4)	upgrade hooks between module versions for its own stored state (no database migrations: state files and caches only)	VaR date caches rebuilt on a format change

The Java SPIs are discovered with `ServiceLoader` **inside the module's class loader** (`META-INF/services/…` in the module's jars) and, for Spring beans, through the module's own child application context (section 4.4).

2.4 What stays kernel-only, and why

Kernel-only	Why
Connector plugins (`SourcePlugin`), storage providers, the lake formats	the site's data, credentials and network belong to the site's administrators; one place to secure and audit them (G4)
Identity, RBAC, the licence check, the registry's model	one trust point (ACCESS_MODEL.md section 2)
The Python runtime, its lock file and sandbox	one runtime (ARCHITECTURE.md section 29.1)
The console shell, menus, Help centre, search, the Elements core	one look, one search, one Help
The database schema files	no migrations, two complete schema files; modules keep state in files under their data folder or in kinds

A new connector type (say, a genomics VCF reader) is a **kernel extension**: a connector plugin jar shipped with the kernel or added to the kernel's `plugins/` folder by the site, signed by the kernel's publisher key; never part of a `.dmod`. A module that needs such a format says so in its data needs (`formats: [vcf]`), and Admin → Data bindings shows *no connector of type vcf is installed*.

2.5 REST namespaces

Prefix	Owner
`/api/v1/{admin,bi,pivot,tables,packs,sutras,views,entities,search,explain,approvals,changes,export,flight,modules,functions,…}`	kernel (the about 100 prefixes of today)
`/api/v1/<api>/…`	a module, `<api>` declared in its manifest (`api.prefix: risk` for banking); unique across installed modules, refused when it clashes with the kernel's list or another module's
`/api/v1/admin/modules`, `/api/v1/admin/bindings`	kernel: module and binding administration (sections 5, 7)
`/m/<module>/static/<version>/…`	console: a module's static files (section 4.7)
`/m/<module>/…` pages	console: a module's pages (declared routes, section 9.8); a declared `alias:` keeps a current address (today's `/var`, `/exposure`), checked against the reserved names, refused on clash
`/m/<module>/api/…`	console: the kernel's API proxy to `/api/v1/<api>/…`, declared endpoints only, as the person (section 9.5)

2.6 Events

One kernel event bus (in process, gossiped over `drishti-net` where marked). Modules subscribe through `ModuleContext.events()`; a module may publish only events under its own id (`banking.*`).

Event	Payload	Typical module use
`data.loaded`	lake, table, kind, business date, batch id, rows	VaR rebuilds a date when `scenario-pv` or `pnl-vector` lands
`data.changed`	kind, ids or range, cause (correction, restatement)	exposure recomputes a netting set
`binding.changed`	kind, old and new source	VaR drops caches for that kind
`date.rolled`	new business date	hot file prepared
`module.started` / `module.stopped` / `module.refused`	id, version, reason	a dependent module reacts
`settings.changed`	module, keys	VaR reloads hierarchies
`approval.decided`	item, decision	regulatory sign-off
`cache.invalidated`	group	—
`function.switched`	word, on/off	AI tools refreshed

2.7 How the kernel is versioned and shipped

- The kernel ships as `drishti-kernel-<product version>.jar` (the thin jar, P2) with its connector plugins, the console, the Elements core and the kernel's content. Its `kernelApi` and `serves` are in `/public/about`.
- A **distribution build** (section 6.6) is the kernel plus the `.dmod` files the distribution names, in one archive; the modules stay separate files inside it.

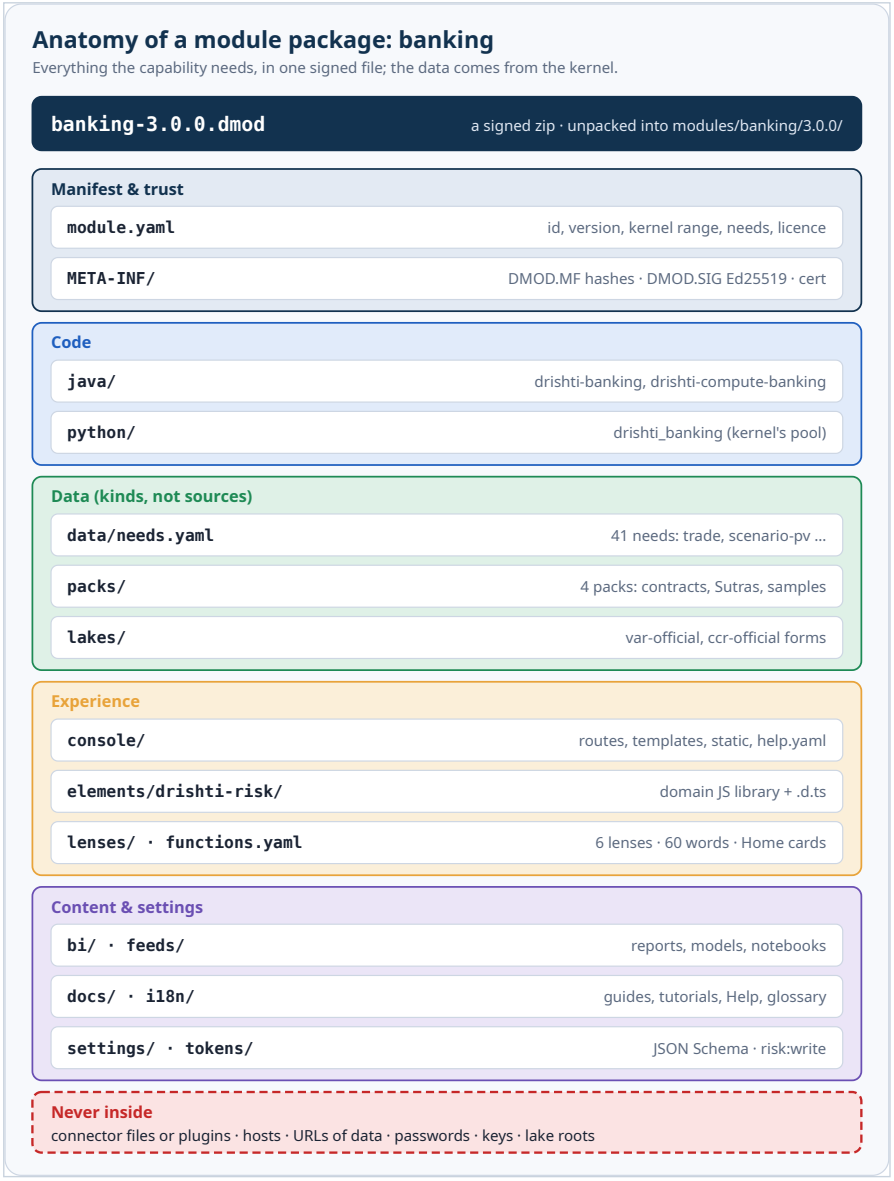
---

3. The module package (`.dmod`)

3.1 File format

- A **zip** (ZIP64 allowed) named `<id>-<version>.dmod`, e.g. `banking-3.0.0.dmod`.
- Signed like a JAR, with a modern signature: `META-INF/DMOD.MF` lists every entry with its SHA-256; `META-INF/DMOD.SIG` is an **Ed25519** detached signature over `DMOD.MF`; `META-INF/publisher.pem` is the publisher's certificate, chained to a key the site trusts (section 11.1). An entry not listed, or listed with another hash, refuses the whole module.
- Uncompressed `module.yaml` first in the archive, so `module info` reads it without unpacking.
- No executable native binaries (`.exe`, `.so`, `.dll`, `.dylib`) at the top level of `python/` or `console/` (Windows QA rule: a module runs on Java and the kernel's Python only); native code a jar bundles (JavaCPP) is allowed and declared (`native: true`), and refused on a site that forbids it.
- Installed **unpacked** into `<modules.dir>/<id>/<version>/` (read-only to the server process), with the original `.dmod` kept beside it for verification at every start.

3.2 Folder layout




```
banking-3.0.0.dmod
├── module.yaml                  the manifest (section 3.3)
├── META-INF/                    DMOD.MF (hashes) · DMOD.SIG (Ed25519) · publisher.pem
├── java/                        the module's jars (own class loader) and their private third-party jars
├── python/                      its Python package(s) for the kernel's pool; requirements.txt (must be within the kernel's lock)
├── data/needs.yaml              the kinds it needs and produces, with contract ranges (section 5)
├── packs/<pack>/                its packs, exactly today's pack folders (pack.yaml, contracts/, sutras/, samples/, python/, guides/)
├── engines/                     engine catalogue entries and engine defaults
├── lakes/                       its published-results lake forms (var-official, ccr-official): tables only, no location
├── bi/                          reports, models, tables, notebooks, fragments, report definitions
├── lenses/                      lenses.yaml (registry lenses: modules, functions, cards, chips) and lens files (menus, Home, Help)
├── functions.yaml               its command words and the APIs behind each
├── console/                     plugin.yaml, routes/, templates/, static/ (drishti_*.js, drishti_*.css), help.yaml
├── elements/<lib>/              its domain Elements library (drishti-risk): ES modules, CSS on tokens, .d.ts, manifest
├── feeds/                       feed definitions and their refresh scripts (.py, with .sh and .ps1 launchers)
├── settings/                    schema.json (JSON Schema 2020-12) and defaults.yaml
├── tokens/token-scopes.yaml     its API token scopes
├── docs/                        guides/, tutorials/, img/, help pages (help.yaml groups), glossary.yaml, README.md
├── i18n/                        message bundles
├── tests/                       its conformance tests (run by `module verify --deep`; never loaded by the server)
└── NOTICES/                     LICENSE, THIRD-PARTY-NOTICES.md
```

The full tree for banking, every folder filled from today's files, is examples/banking/dmod-tree.txt.

3.3 The manifest (`module.yaml`)

The manifest grows from today's `META-INF/drishti/module.yaml` (id, domain, title, feature, requires, contributes); it is still read only by the registry (other kernel parts ask the registry). Every field:

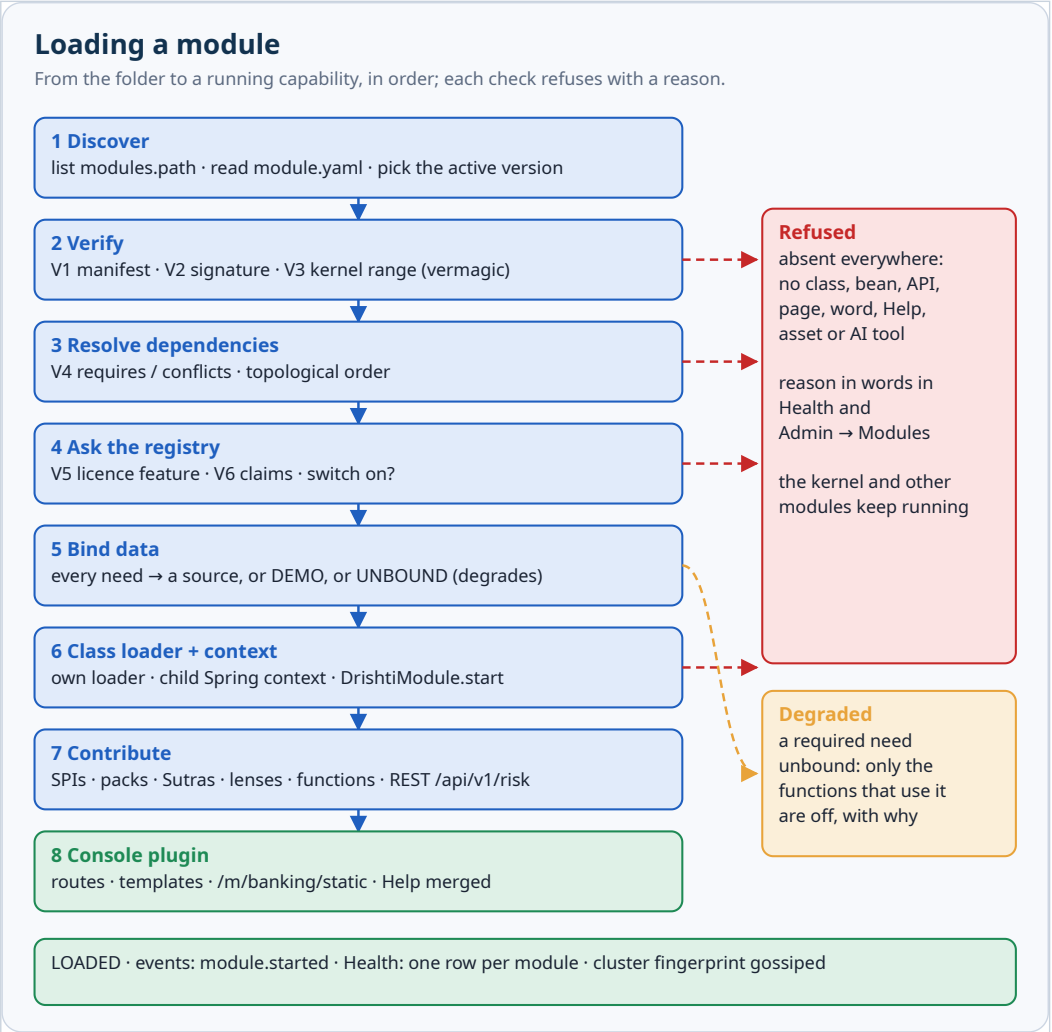
Field	Required	Meaning
`format`	yes	manifest format, `1`
`id`	yes	lower-case id, unique (`banking`); also the folder, the class-loader name, the events prefix
`version`	yes	semver of the module (`3.0.0`)
`title`, `about`	yes	words for Admin, About, Help
`domain`	yes	the domain it serves (`banking`, `economy`, `capitalmarkets`, `genomics`)
`publisher`	yes	`{name, id, contact}`; must match the signing certificate's subject
`licence.feature`	yes	the licence feature that unlocks it (`banking`); `licence.limits` it understands (`positions`, `nodes`)
`kernel`	yes	the kernel API range (`">=1.0 <2.0"`)
`rachana`, `contracts`, `plugin-api`, `elements`	yes	grammar, contract-format, console plugin API and Elements core ranges it was built for
`requires`	no	other modules with ranges, and `optional:` ones it uses when present (`capitalmarkets` optional for banking)
`conflicts`	no	modules it cannot run beside
`provides.kinds`	no	kinds it owns (defined in its packs), by pack
`java`	no	`jars:` (in `java/`), `entry:` the `DrishtiModule` class, `spring:` its auto-configuration, `exports:` packages other modules may *not* see (default none), `native:` true when a jar bundles native code
`compute`	no	its engines (ids, words, pools, switches)
`python`	no	`packages:`, `requirements:`, `pools:` (the kernel pools it submits to, `python` by default), `items:` (Python compute items it registers)
`packs`	no	its packs (folders under `packs/`) and which are on by default
`data`	no	`needs:` file (`data/needs.yaml`, section 5)
`api`	no	`prefix:` (its REST namespace), `aliases:` (old paths answered for one release), `tokens:`
`console`	no	`plugin:` (`console/plugin.yaml`, format 2: routes, aliases, words, menus; section 9), `elements:` libraries it ships
`functions`	no	`functions.yaml` (or inline): words, their APIs, their module
`lenses`, `home`	no	`lenses/lenses.yaml`; Home cards and chips per lens
`feeds`	no	its feed definitions (each fills one of its kinds)
`bi`	no	its BI folders
`docs`	no	Help sections and pages, guides, tutorials, glossary; `features:` rows for the docs register
`settings`	no	`schema:` and `defaults:`
`events`	no	events it publishes and subscribes to
`parts`	no	the parts the site places separately, each with its resource class (`light`, `heavy` with `memory-mb`, `feed`, `singleton`) and what it contains (section 4.11.2)
`resources`	no	expected memory and threads (the capacity planner and the loader's budget check)
`health`	no	its health checks' ids
`taint`	no	set by the kernel, never by the publisher

The full banking example: examples/banking/module.yaml. It lists the two jars (`drishti-banking`, `drishti-compute-banking`), the six engines, the Python package `drishti_banking` with its items, the five packs (with `banking-reference`, open question Q1), the 30-odd data needs, the console plugin and `drishti-risk`, the six lenses, 60 functions, the Home cards, the market-data feed (open question Q6), the seven guides and nine tutorials, the settings schema, the licence feature `banking`, the optional `capitalmarkets` dependency and the kernel range.

Representative stub files with real-looking content:

File	Shows
data/needs.yaml	banking's data needs (section 5.3)
settings/schema.json	the settings schema (VaR windows, confidence, root names, hierarchies)
packs/market-risk/sutras/var-desk.v1.sutra.yaml	one Sutra as the module ships it
docs/help/var-in-five-minutes.md	a Help page stub
console/plugin.yaml	the console plugin as packaged (today's file, paths made module-relative)

4. Loading



4.1 Module paths

Setting	Default	Meaning
`drishti.modules.path` (`DRISHTI_MODULES_PATH`)	`${drishti.data.dir}/modules`	an ordered list of folders holding installed modules (`<id>/<version>/` unpacked + the `.dmod`); first match of an id wins; read-only to the server
`drishti.modules.inbox`	`${drishti.data.dir}/modules-inbox`	where `module install` and Admin → Modules put an uploaded `.dmod` waiting for approval
`drishti.modules.active`	`config/modules/active.yaml`	which version of each installed module is active (written by install, upgrade, rollback; four eyes)
`drishti.modules.trust`	`config/trust/publishers/`	publisher certificates the site trusts
`drishti.modules.<id>.enabled`	`true`	the existing switch (ACCESS_MODEL.md section 8)

All folders live under the configurable data folder (no root litter, visible names); the server never writes to a module's folder.

4.2 Discovery

At start, before any bean: list every `<path>/<id>/<version>/module.yaml`; pick the version `active.yaml` names (else the highest); parse each manifest against the manifest schema; build the dependency graph. Nothing else of a module is touched yet.

4.3 Verification, in order

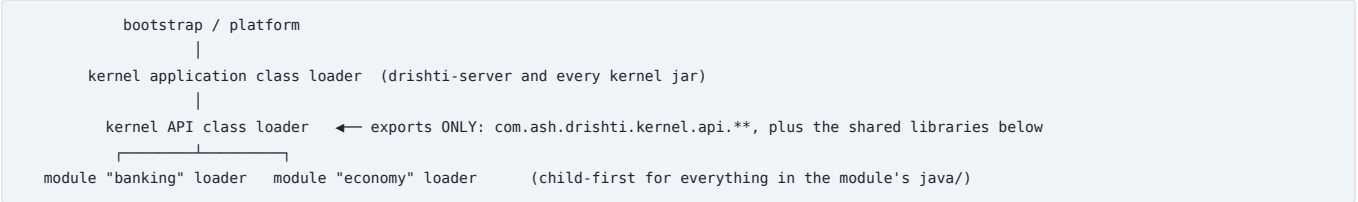
Each step refuses with a reason code and words; a refused module is **absent everywhere** (section 4.9).

#	Check	Refusal (example words)
V1	manifest valid (schema, `format: 1`, id = folder)	*banking 3.0.0: module.yaml line 12: `kernel` missing*
V2	signature: `DMOD.MF` hashes match the unpacked files and the `.dmod`; `DMOD.SIG` verifies; certificate chains to a trusted publisher; publisher id = manifest's	*signature does not verify (file java/drishti-banking-3.0.0.jar changed)*
V3	kernel range (`vermagic`): `kernel`, `rachana`, `contracts`, `plugin-api`, `elements` ranges contain the kernel's	*built for kernel API >=2.0; this kernel is 1.4*
V4	dependencies: every `requires` present, in range, and itself not refused; no cycle; no `conflicts`	*needs capitalmarkets >=1.2; 1.1 installed*

#	Check	Refusal (example words)
V5	licence through the registry: the licence names the feature, is valid on this date, within limits	*the licence does not include banking*
V6	claims: its API prefix, paths, functions, kinds and lens ids clash with no kernel or other module's	*function VAR is claimed by banking and riskx*
V7	data needs: every `provides.kinds` contract parses; every `needs` contract range is satisfiable by some module or kernel kind	a missing *binding* does not refuse: it degrades (section 5.6)
V8	resources: its declared memory fits the node's budget	*needs 6 GB; 3 GB free in the module budget* (warning, or refusal when `drishti.modules.strict-resources`)
V9	enabled: the site switch	not a refusal: *off by the administrator*

Order: topological by `requires` (a module after everything it requires), then by id for a stable order. A refusal propagates to every module that requires it (*refused: requires banking, which is refused*).

4.4 Java: one class loader per module



- **Shared (parent-first)**: `com.ash.drishti.kernel.api..`, the Java platform, Spring Framework and Boot, Jackson, SLF4J, Apache Arrow, Micrometer, Jakarta Servlet and Validation. The list is part of the kernel API (a MINOR may add, only a MAJOR removes).
- **Private (child-first)**: every other class in the module's `java/` jars, including its own copies of libraries (a module may ship commons-math 3.6 while the kernel uses another version).
- **Hidden**: every kernel class outside the API (`com.ash.drishti.server..` etc.) is not visible to a module (`ClassNotFoundException`); an architecture test in the SDK fails the build earlier.
- **Spring**: each module gets a **child** `ApplicationContext` with the module's class loader; its parent exposes only the kernel API beans (the `ModuleContext` services). Its `@ConditionalOnModule` stays as a guard. SPI implementations are beans or `ServiceLoader` entries in that context; the kernel collects them through the registry of contributions (`ModuleContributions`), never by scanning a module's packages.
- **No module sees another module's classes**; modules talk through kinds, functions and events.
- **Stop**: `DrishtiModule.stop()`, then the child context closes, the contributions are withdrawn, the class loader is dropped (P5 hot unload depends on this being leak-free; a leak test loads and unloads 50 times in the SDK harness).

4.5 REST

The module's controllers (`@ModuleApi`) are mounted by the kernel under `/api/v1/<prefix>/`, behind the kernel's filters in this order: TLS/identity → module gate (`DRS-1025` when off) → function gate (`DRS-1026`) → permission (`DRS-1027`) → audit → the module's `DispatcherServlet` (one per module, bound to its child context). A module cannot register a filter, a servlet or a path outside its prefix. `ApiAlias` keeps working for one release.

4.6 Python compute

- The module's `python/` packages are made importable **only in that module's runs**: a run submitted by `banking` gets `<module>/python` prepended to its sandbox's import path; another module's runs never see it. The kernel's `drishti_kernel` package and the locked libraries are shared.
- **No pip at install time.** `python/requirements.txt` must be satisfied by the kernel's lock file; a module needing a library outside it is refused with *needs scipy-stats-x 1.2, not in this site's Python lock: an administrator approves it first* (ARCHITECTURE.md section 29.6).
- Every run goes through the kernel's one door (`PythonRuns`): same strict sandbox, limits, audit line, the person's row rules on every frame. The audit line gains `module`.
- A module's Python items (BI python visuals, Python measures, its compute packs) are registered by the manifest's `python.items`, shown in Build with the module's name.

4.7 The console plugin

- **Where**: the console reads the same `modules.path` (or its own `modules.path` pointing at the same folders: one cluster setting). For each module the server reports as *loaded* (`/public/about`), the console mounts `<module>/console/`.
- **`plugin.yaml` format 2**: everything the console shows for a module is declared (routes, the API proxy list, words, menus, Home cards, slots, settings page); format 1's `routers:` (FastAPI routers included at import) and the shared template loader go. Section 9 is the design; format 1 is read for one release with a deprecation line, then refused.
- **Python code** (optional page handlers only): imported as `drishti_modules.<id>` (a namespace package rooted at the module folder), not `plugins.<id>`, lazily, on a route's first use; it registers nothing.
- **Templates**: one sandboxed Jinja environment per module, namespace `<id>:`, which sees only the kernel's public layouts and its own files; a kernel template never includes a module's (rule 1) and a module never overrides a kernel template (section 9.2, 9.3).
- **Static files**: served from `/m/<id>/static/<version>/…` with a year-long cache (the version busts it) and SRI, only to people for whom the module is on and their lens uses it (today's `_module_asset_refused`, unchanged). The `drishti_` prefix stays (section 9.7).
- **Elements**: the module's `elements/<lib>/` is added to the Elements manifest with `module: <id>`; the loader's existing refusal applies.
- **Help merge**: `console/help.yaml` heads and pages (today's `help-sections:` / `sections:`) merge into Help; the guides and tutorials are read from the module's `docs/` folder; the docs register gains the module's `docs.features` rows, so the docs gates check a module's docs like the kernel's.
- **Sitemap and menus**: module menu entries carry `requires: module:<id>` as today; lens files from `lenses/` replace `config/lenses/<module>/`.

4.8 Order of start

1. Kernel core (settings, registry, licence, identity).
2. Discovery and verification of modules (4.2, 4.3).
3. **Data bindings resolved** for every need (section 5): before any module starts, so a module starts knowing which of its needs are bound.

4. Modules started in dependency order: class loader, child context, `DrishtiModule.start`, contributions registered.
5. Packs loaded (kernel packs, then modules' packs in module order), Sutras, lenses, functions.
6. Kernel services open (REST, console, scheduler, feeds); `module.started` events.

A module's start has a time budget (`drishti.modules.start-seconds`, 120); exceeded → refused (*did not start in 120 s*), the kernel goes on.

4.9 Failure: refused means absent

A refused module (any of V1–V8, a start failure or a crash in `start`) is treated exactly as *not installed*: no class loaded (its loader is dropped), no bean, engine, job, API, pack, connector binding, route, menu, search hit, command, Help page, asset, AI tool, export or embed. What remains:

- **Health**: one row per module: `LOADED`, `LOADED (degraded: 2 data needs unbound)`, `OFF (administrator)`, `REFUSED (reason)`, `TAINTED`.
- **Admin → Modules**: the same with the full reason, the check that failed, and what to do.
- **Words for people**: a typed function of a refused module answers *VAR is not available: the banking module is not running (signature does not verify). Ask your administrator.* (administrators see the reason; others see *not available*).
- **Kernel keeps running**; a refused module never stops the kernel. A refused *required-by-distribution* module (section 6.2, `required: true`) makes Health `DEGRADED` for the whole instance.

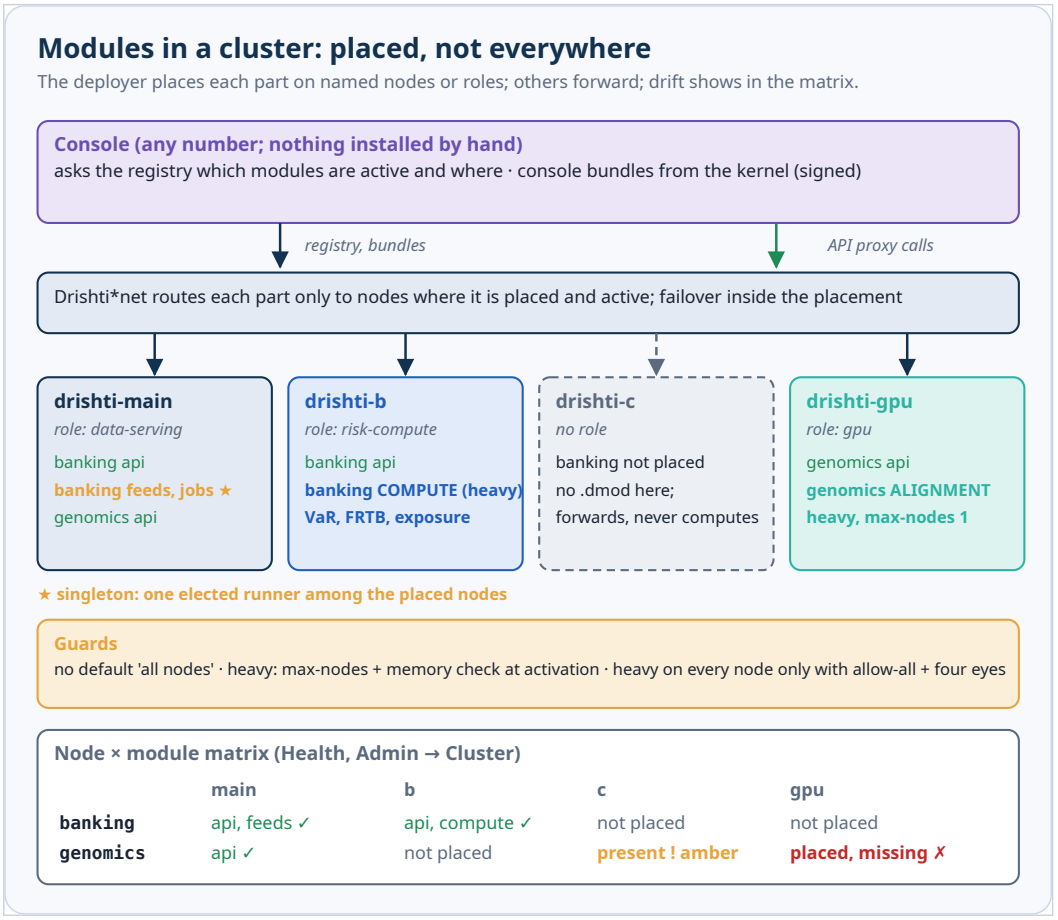
4.10 Restart versus hot load

What	P2	P5
Packs, Sutras, lenses, Help, BI content inside an installed module	**hot** (the pack reloader, as today)	hot
Settings of a module	hot where the schema marks a key `hot: true`; else *restart needed*	same
Data bindings	hot (section 5.8)	hot
A module's pages, menus, words, Help (declared in `plugin.yaml`)	**hot** through the live route table (section 9.8): off, on and **remove** always; install and upgrade when the console part is declarative and the server has the module loaded	hot
A module's code (Java, Python package, console Python handlers)	**restart**: install/upgrade/remove stages the change, Admin shows *restart needed*, a rolling restart applies it	**hot**: unload (stop, close context, drop loader), load the new version; refused when a leak check fails
A module's static assets	with the code (versioned path)	same

The development build: the SDK's harness and the developer's IDE run the kernel with a module folder pointing at the module's build output (`target/dmod/`), same loader, same checks, a development key (section 8.5). There is no "on the class path" second path.

4.11 Modules in a cluster

The owner's question: *what happens when a module is deployed on one node and not on another, and how does the console deal with it?* The answer: **the deployer decides which nodes get a module, part by part**. There is no default "all nodes": a module runs only where it is placed, heavy work never spreads by itself, every other node forwards, and the console never talks to a node's module, only to the cluster's registry and the kernel's API proxy. Drishti*net itself (membership, gossip, the capability digest, routing, roles and tags) is CLUSTER_GUIDE.md section 1; this section adds only what modules need.



4.11.1 Explicit placement, opt-in

- A module is installed on **the nodes the deployer names**, by **role** (a node tag, `role:risk-compute`, as in `config/net/node.yaml`) or by **node name** (`drishti-b`). **There is no default "all nodes"**: a module with no placement is installed nowhere and shows *not placed* in Admin → Modules.
- **One home**: `config/placement.yaml` (cluster configuration, four eyes to change, audited), edited by hand or in Admin → Modules → *Placement* (a node picker per part, showing each node's role, free memory and current modules). The example: examples/site/placement.yaml.
- The kernel **copies the `.dmod` only to nodes where at least one part is placed**; other nodes never hold it.

4.11.2 Placement per part, with resource classes

The manifest declares the module's **parts** and each part's **resource class**; the deployer places each part separately, so heavy parts go on fewer nodes than light ones.

```
# module.yaml (the publisher declares the parts; never where they go)
parts:
  api:      { class: light,     contains: [rest, view-augmenters, explain, functions] }   # the façade
  compute:  { class: heavy,     contains: [engines: [var, frtb-sa, simm], kind-providers, python-items], memory-mb: 6144 }
  exposure: { class: heavy,     contains: [engines: [exposure]], memory-mb: 4096 }
  feeds:    { class: feed,      contains: [feeds] }                                        # always singleton
  jobs:     { class: singleton, contains: [scheduled-jobs, official-results-publish] }
  console:  { class: light,     contains: [console-bundle, help, glossary] }              # served to every console via the registry
```

Class	Meaning	Guard
`light`	registry entries, the REST façade, Help, the console bundle	may be placed widely
`heavy`	compute engines, `KindProvider`s, Python items; declares `memory-mb`	a **max-nodes** limit set by the deployer; a **memory headroom check** on each placed node at activation (refused with the reason: *drishti-c: banking.compute needs 6,144 MB, 2,300 MB free in the module budget*); Admin warns before it goes wide
`feed`, `singleton`	feeds, schedulers, daily jobs, publishing official results	**exactly one runner**: leader election among the placed nodes (the lease mechanism the inbox and run leases already use); failover to another placed node only

Banking (a 3-node site): `api` on `drishti-main` and `drishti-b`; `compute` and `exposure` on `drishti-b` only; `feeds` and `jobs` on `drishti-main` (singleton); **nothing on `drishti-c`** (its `.dmod` is not even copied there). **Genomics**: `alignment` compute (class heavy, `gpu: true`) on the one GPU-class node (`role:gpu`), its API on the same node and `drishti-main`.

4.11.3 Guards against heavy work spreading

- **Singletons run once**, by election among their placed nodes, never on all.
- **Heavy parts** obey the deployer's `max-nodes` and the per-node memory check at activation.

- **A node never computes for a module part it is not placed for**: routing sends work only to placed, active nodes; an unplaced node that receives a module request (a client hit it) **forwards** it, never computes it.
- **Activation refuses a placement that puts a heavy part on every node** (*banking.compute on 3 of 3 nodes: confirm with allow-all*), unless the placement says `allow-all: true` and a second administrator approves it.
- **Health shows the effective placement**: per module and part, the nodes placed, active, leader (for singletons) and the memory used.

4.11.4 Install and activation across the placed nodes

- **One action**: a deployment (section 7.6: `drishti.py module deploy … --plan` / `--apply`, or Admin → Modules → *Deploy*), approved once (four eyes). No per-node install, no manual copying.
- **Replication to placed nodes only**, over Drishti*net (chunked, mutual TLS, the cluster key); each receiving node checks the SHA-256 and the signature itself (V2 on every node; a node never trusts another's verification). On AWS with EFS the identical path is read.
- **Two-phase activation across the placed nodes**, run by the coordinator: (1) **stage** on each placed node: unpack, run V1–V9 locally (kernel range, licence, claims, the memory check for the parts placed there), answer *ready* or *refused (reason)*; (2) **activate them together** when all answered *ready* within `cluster.modules.stage-seconds` (300): the cluster configuration moves to version N+1 and each placed node applies it (hot for declarative parts, at the rolling restart for code in P2: section 4.10), or **abort on all of them** when one refused: staged files removed, nothing changed, the proposer sees which node and why.
- **A late joiner** receives **only the modules placed on it** (by its name or its role) and serves them only after activation; a node given a role later is staged and activated the same way.

4.11.5 Routing and availability

- Drishti*net routes a module's API calls, compute and jobs **only to nodes where that part is placed and active** (each node's capability digest gains a `modules` part: id, version, the parts active here, the singleton leader). **Failover stays inside the placement**: the next placed node, never an unplaced one.
- **No placed node up** for a part: the part is **unavailable**, not absent. People see *Banking's VaR is unavailable right now (no compute node is up); try again later*; pages that need only the façade still work; saved artefacts, schedules (paused) and data stay; nothing is tombstoned. Absent (section 10) is only for an off or removed module.

4.11.6 Drift

- Each node gossips its **module fingerprint** (`id@version#sha256-of-DMOD.SIG`, with the parts it runs); drift is **a node's actual state differing from its placement**.
- Health and Admin → Cluster show a **node × module matrix**: **green** placed and active; **red** placed but missing, refused or at another version (that node is left out of routing for the module until *Resync* re-sends the package and activation passes); **amber** *present but not placed* (a leftover `.dmod`: the node serves nothing for it; *Clean up* removes it).
- In strict mode (`cluster.modules.consistency: strict`, the default) a node whose **kernel API range** differs is refused from joining: modules cannot be checked against a kernel the cluster does not run.

4.11.7 How the console interacts

- **The console never talks to "a node's module".** It asks the **cluster registry** (through its entry node, `backend.urls` with discovery as today: CLUSTER_GUIDE.md section 1.8) which modules are active and where.
- **The console bundle is a light part served by the kernel to every console**: `GET /api/v1/modules/<id>/bundle/<version>` streams the `console/`, `elements/` and `docs/` parts; the console checks them against the signed `DMOD.MF`, caches them under `console-cache/modules/<id>/<version>/` and serves from there. A separate console machine needs **no manual install**, and several consoles serve identical bytes. Its pages exist on every console; **its API calls still route only to placed nodes**.
- **API calls** go through the kernel's API proxy (section 9.5) to the entry node, which routes by capability as in 4.11.5.
- **The live route table follows the registry's events** (`module.activated`, `module.unavailable`, `module.available`, `module.removed`): unavailable parts show the notice above, removal shows the notices of section 10.4.

4.11.8 Rolling upgrade and rollback

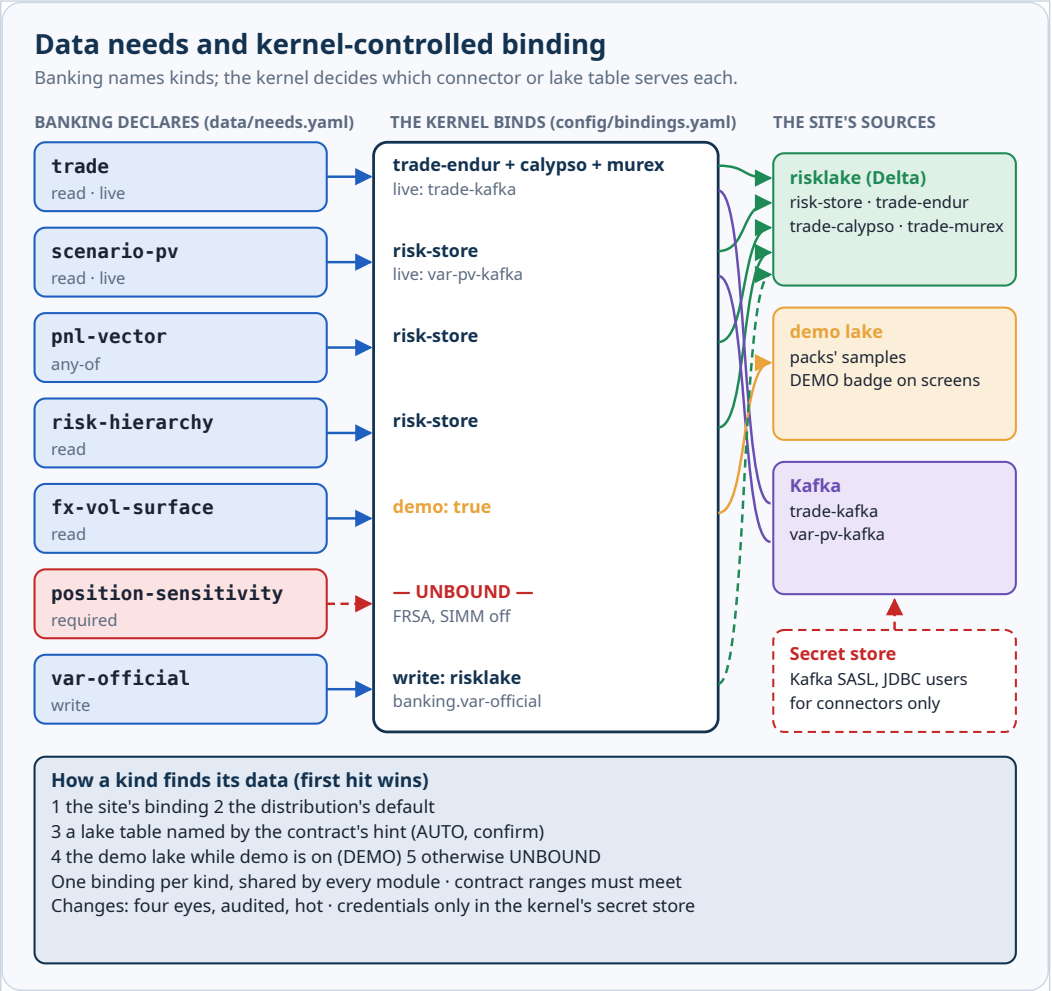
- For a short window (`cluster.modules.upgrade-window-minutes`, 30) two versions may run **across the placed nodes** when both are inside each other's compatibility range. Each placed node advertises its version.
- Requests are **pinned per session** to one version: the proxy routes the session only to placed nodes with that version, and the console serves the matching bundle (`/m/<id>/static/<version>/` lets both coexist), so a page always talks to the API version it was built for. When the old version's last placed node is upgraded, pinned sessions get the *updated: reload* notice.
- The placed nodes then converge; rollback is the same in reverse. A mixed set outside the window is drift (red).

4.11.9 Tests

Test	Proves
Placement honoured	on three nodes with banking placed on main and b: the `.dmod` exists only on main and b; c holds nothing of it; the matrix shows exactly that
Heavy work stays put	1,000 VaR requests through each node (including c): every computation runs on b; c and main forward, never compute (engine counters, thread names `mod-banking-…`)
Singleton once	feeds and daily jobs run once per schedule; kill the leader: another placed node takes over; never an unplaced node
Failover inside the placement	stop b: VaR is *unavailable* with the notice (not absent, not computed on main or c); saved work intact; restart b: available again
All-nodes heavy refused	placing `compute` on 3 of 3 nodes is refused without `allow-all: true` and a second approval
Memory check	a heavy part placed on a node without headroom is refused at stage, and the activation aborts on all placed nodes
Late joiner	a fourth node with `role:risk-compute` receives only banking (placed by role), activates it, then serves; a fourth node without the role receives nothing
Drift	delete the module on b: red cell, b out of routing, *Resync* repairs; copy a `.dmod` onto c: amber cell, c serves nothing for it
Rolling upgrade	3.0.0 → 3.0.1 across the placed nodes under load: sessions stay pinned until reload, no request reaches a mismatched API, no failed request
Console without install	a separate console shows a newly placed module's pages from the kernel-served bundle; two consoles serve identical bytes; its API calls land only on placed nodes

5. Data needs and connector binding

The heart of requirement 4: **a module says what data it needs; the kernel decides where that data comes from.**



5.1 Who owns what

Thing	Owner	Where
Connector **types** (plugins)	kernel	`plugins/drishti-plugin-*`, the kernel's plugin folder
Connector **configurations** (a Kafka cluster, a JDBC database, a Delta root on S3)	the site's data administrator	`config/connectors/<name>.yaml` (unchanged), credentials as references `${secret:…}`
Credentials	the site's security administrator	the kernel's secret store (environment, a secrets file outside `config/`, or an external vault)
Lakes and their tables	the site	`config/lakes/<lake>.yaml` (unchanged)
Kinds and their contracts	the module that defines them (`provides.kinds`), or the kernel (core kinds)	the module's packs (`packs/<pack>/contracts/*.contract.yaml`)
Data needs (which kinds a module reads or writes, which contract versions, required or optional)	the module	`data/needs.yaml` in the `.dmod`
Bindings (kind → which connectors / lake tables serve it, in what read order, live or not, and where produced kinds are written)	the site's data administrator, **through the kernel**	`config/bindings.yaml` (new) + Admin → Data bindings, four eyes
Default bindings for a product	the distribution	`bindings.defaults` in the distribution file (section 6)

5.2 Contracts: the format the kernel owns

The contract format is today's `*.contract.yaml` (kind, mnemonic, pack, version, class, grain, key, dated-by, cadence, fields with type and presence, vectors) with **one change**: the `storage:` block (which today names `connector: risk-store`, `plugin: delta`, `table:`) is **removed from the contract** and replaced by a connector-free **hint**:

```
binding-hint:                # what a site usually binds; never a connector name, plugin or location
  lake-table: risk.scenario-pv          # the table name a lake file would use
  partition: [BUSINESS_DATE]            # the first partition is always the business date
  live: optional                        # a live source makes sense (intraday)
  demo: samples/scenario-pv.jsonl       # the demo lake's sample for this kind
```

The kernel reads `contract-format: 1`; a module declaring `contracts: ">=1 <2"` is checked at V3.

5.3 Data needs (`data/needs.yaml`)

Each need: the kind, the contract version range, `use` (`read`, `write`, `live`), `required` (true / false), `any-of` groups, the functions and engines that depend on it, and its `demo` allowance. Banking's needs, abridged (the full file: examples/banking/data/needs.yaml):

Kind	Contract	Use	Required	Used by	Notes
`trade`	`>=1.0 <2`	read, live	yes	TRD, VAR (scope by trade), EXP, PREDEAL	a union of trade stores (Endur, Calypso, Murex)
`scenario-pv`	`>=1.1 <2`	read, live	any-of A	VAR, PVAR, SMAT	per trade, own currency; intraday stream
`pnl-vector`	`>=1.0 <2`	read	any-of A	VAR, PVAR, SMAT	instead of scenario PVs
`fx-scenario`	`>=1.0 <2`	read	with scenario-pv	VAR	converts scenario PVs to USD
`scenario-set`	`>=1.0 <2`	read	yes	VAR, SWIN	the 509-day and 250-day windows
`risk-hierarchy`	`>=1.0 <2`	read	yes	VAR, VLIM, HIER	the VaR tree and limit letters
`portfolio`	`>=1.0 <2`	read	no	VAR PORT	
`var-limit`	`>=1.0 <2`	read	no	VLIM, limit bars	
`desk-pnl`	`>=1.0 <2`	read	no	backtesting	actual and hypothetical P&L
`pnl-history`	`>=1.0 <2`	read	no	SWIN	stress window selection
`position-sensitivity`	`>=1.0 <2`	read	yes for FRSA, SIMM	FRSA, SIMM, SENS	
`risk-factor`	`>=1.0 <2`	read	yes for FRSA, SIMM	RFAC	
`netting-set`, `csa`, `agreement`, `collateral-balance`	`>=1.0 <2`	read	yes for EXP	EXP, PREDEAL, SIMM	
`credit-limit`	`>=1.0 <2`	read	no	limit use	computed use, stored limits
`ir-curve`, `fx-spot`, `fx-vol-surface`, … (market data)	`>=1.0 <2`	read	yes for EXP; no for VaR	EXP paths, TVAL	from capital markets when installed (Q6)
`counterparty`, `legal-entity`, `book`, `desk`, `trader`	`>=1.0 <2`	read	yes	every screen	`banking-reference` (Q1)
`calendar`	kernel `>=1.0`	read	yes	business dates	kernel kind
`var-official`, `ccr-official`	own	**write**	no	VOFF, XOFC, official results	written to a lake the site binds
`position`, `var-result`, `exposure-result`	own	**computed**	—	POS, VRES	`KindProvider`s; not bindable

5.4 Bindings (`config/bindings.yaml`)

Bindings are per kind, not per module. A kind is one dataset at a site; every module that needs it reads the same binding. The file (examples/site/bindings.yaml) has three example sites in one:

```
bindings:
  trade:
    read: [trade-endur, trade-calypso, trade-murex]   # connectors (config/connectors/<name>.yaml), in read order
    union: true                                        # one trade store across them
    live: trade-kafka                                  # live updates (a Kafka connector)
  scenario-pv:
    read: [risk-store]                                 # a Delta connector on risklake (lake: risklake)
    live: var-pv-kafka
  risk-hierarchy:
    read: [risk-store]
  var-official:
    write: { lake: risklake, table: banking.var-official }
  pnl-vector:
    demo: true                                         # explicitly the demo lake (a test site)
```

Key	Meaning
`read`	connectors that serve the kind, in read order (today's `readOrder`: Redis for recent days, Delta behind it)
`union`	rows from all of them (today's `union: true` on the trade connectors)
`live`	a connector that pushes changes (Kafka, RabbitMQ, ActiveMQ); subscribers get them through `Kinds.subscribe`
`write`	where a produced kind is written: a lake and table (the kernel's writer; the module never names a path)
`demo`	`true` binds the demo lake's samples explicitly
`for:` (exception)	a per-module override, `for: {genomics-qc: [qc-store]}`; allowed only for kinds marked `shareable: per-module` in their contract (Q4)

Today's equivalent is spread out: a connector file's `kinds:` list and its `lake:`; the contract's `storage.connector`. P1 folds both into `bindings.yaml` (no dual machinery): connector files keep `plugin`, settings and `lake:`, lose `kinds:`; `drishti.py bindings migrate` writes `bindings.yaml` from today's files once, and a connector file still carrying `kinds:` is read for one release with a deprecation line in Health.

5.5 How a kind finds its data (resolution)

For each kind any loaded module needs, in this order, first hit wins:

1. **The site's binding** in `config/bindings.yaml` (as approved).
2. **The distribution's default** (`bindings.defaults`, section 6), e.g. `drishti.finance` binds banking's kinds to `risklake` tables by their hints when a lake file holds them.
3. **The lake convention**: a configured lake (not demo) whose lake file has the contract's `binding-hint.lake-table` table, read by a connector of the lake's format → bound, marked `AUTO` (Admin shows *found by convention: confirm*).
4. **The demo lake**, only while `drishti.demo.enabled` and the need allows demo: marked `DEMO` (amber everywhere: "demo data").
5. Otherwise **UNBOUND**.

Demo and real never mix (DATA_LAKE_GUIDE.md): a kind is either demo or real; a pack whose kinds are bound to more than one lake is `DEGRADED` (today's warning, moved from packs to bindings).

5.6 What a missing or bad binding does

State	Meaning	Effect
`BOUND`	a source serves it; its schema satisfies the contract range	normal
`AUTO`	found by convention	normal; Admin asks to confirm
`DEMO`	demo samples	works; "DEMO DATA" badge on screens using it
`STALE`	bound, but no load within the contract's cadence	works; Health amber *scenario-pv last loaded 2026-11-04 (cadence daily)*
`MISMATCH`	the source's columns or types break the contract (a required field missing)	**treated as unbound for that need**; Health red with the columns
`UNBOUND`, required	nothing serves it	the module **loads but is degraded**: functions that depend on the need are off with the reason (*VAR needs scenario-pv or pnl-vector: no source is bound; ask your data administrator*); others work
`UNBOUND`, optional	nothing serves it	info only; dependent features hidden (e.g. backtesting without `desk-pnl`)

`any-of` groups are satisfied by one bound member. Health lists `needs[]` per module; `GET /api/v1/admin/bindings` answers the same.

5.7 Computed kinds and the end of module connectors

Today banking ships connector plugins (`var-positions`, `var-pv-stream`) and reactive connectors (`var-engine`, `exposure-engine`). Under G4 these are not connectors: they **compute** a kind from other kinds. They become `KindProvider`s in the module: `position` and `var-result` are *computed kinds* (`provides.kinds[].computed: true`), never bindable, always the module's. The transport part of `var-pv-stream` (reading Kafka) is the kernel's Kafka connector bound as `scenario-pv`'s `live:` source; banking subscribes to the kind. Connector files written for the old names keep loading for one release (today's `moved-classes.properties` mechanism).

5.8 Changing a binding

Admin → Data bindings (mock-up b) or `drishti.py bindings set scenario-pv --read risk-store`: proposed, approved by a second administrator (four eyes), audited, applied **hot**: the kernel validates the new source against the contract, swaps the route, publishes `binding.changed`; modules drop their caches for that kind. A binding change never needs a module restart.

5.9 Two modules, one kind

- One binding per kind: banking and a hypothetical `collateral` module both read the one `trade` binding.
- **Contract ranges must intersect**: if banking needs `trade >=1.0 <2` and `collateral` needs `>=1.3 <2`, the bound source must satisfy 1.3+; V7 refuses a module whose range cannot meet the kind owner's version (*collateral needs trade >=2.0; banking provides 1.4*).
- **One owner per kind**: the module that defines it in its packs; another module that needs it `requires` the owner (optionally) or uses a kernel kind. Two modules defining the same kind id → V6 refuses the second.
- Per-module views of a kind are row rules and masks (permissions), not separate bindings; the `for:` exception is Q4.

5.10 Credentials never inside a module

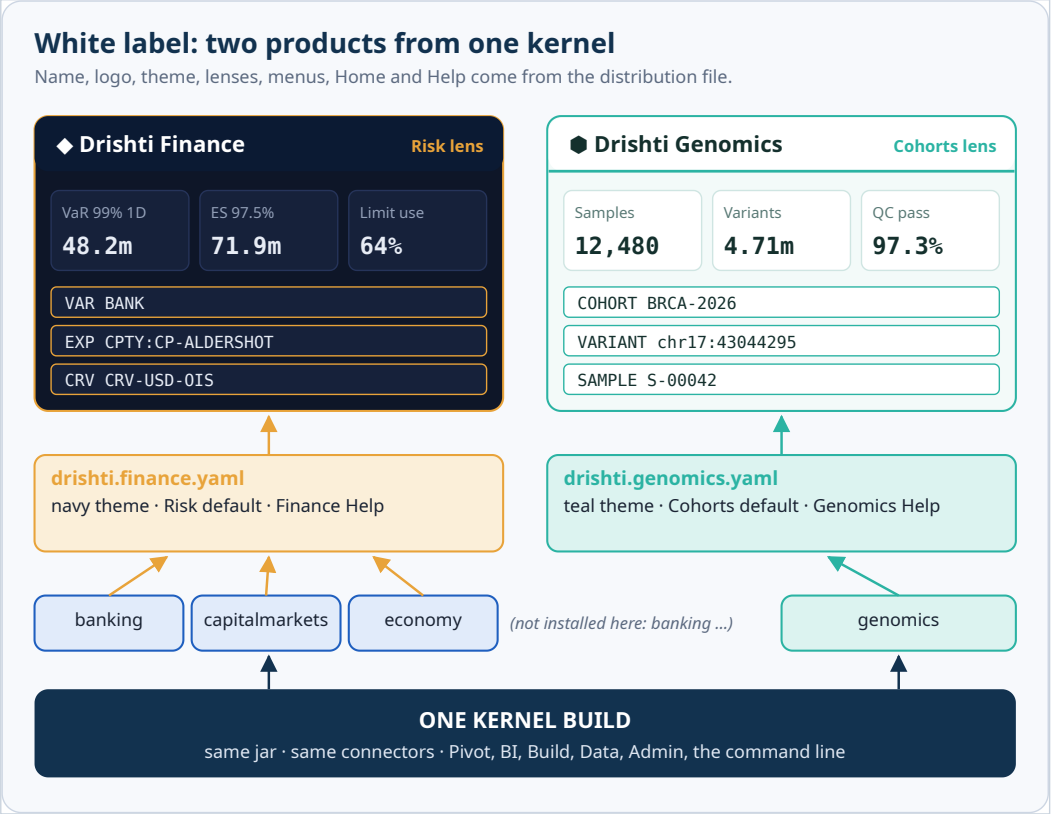
- A module's files contain no connector, plugin, host, URL of a data source, user name, password, key or token: the packager and V1 scan for connector keys (`plugin:`, `connector:`, `bootstrap-servers`, `jdbc:`, `password`, `secret`, `token`) and known secret patterns, and refuse.
- At run time a module has no API to read connector settings or secrets; `Kinds` returns rows, never the connection.
- Feeds a module ships (public sources such as the ECB rates) name a public URL and need no credentials; one that needs a key reads it from the kernel's secret store by a **name the site provides** in the feed's binding (`feeds.market-data.api-key: ${secret:ecb}`), not from the module.

5.11 Banking's example bindings, three sites

Site	`trade`	`scenario-pv` / `pnl-vector`	`risk-hierarchy`	`var-official`
Laptop demo	`DEMO`	`DEMO`	`DEMO`	written to the demo lake
Bank on Delta (`risklake`)	`trade-endur` + `trade-calypso` + `trade-murex` (union) on risklake	`risk-store` (Delta, risklake)	`risk-store`	`risklake` table `banking.var-official`
Bank with live trades	the same + `live: trade-kafka` (Kafka, SASL credentials from the secret store)	`risk-store` + `live: var-pv-kafka`	`risk-store`	the same

6. The distribution file (white label)

6.1 What it is



One YAML file, `config/distribution.yaml` (or `DRISHTI_DISTRIBUTION=<path>`), that turns the kernel into a product: which modules, which brand and theme, which lenses and Home, which docs, which default bindings. It folds today's scattered product settings (`drishti.edition`, `ui.product`, `ui.copyright`, `drishti.branding.*`, the lens files' `title`, `config/packs/default-packs.yaml`) into one place, **as the one way**: the old keys are read for one release when no distribution file exists.

6.2 Fields

Field	Meaning
`distribution`	id (`drishti.finance`)
`version`	the distribution's own version
`kernel`	the kernel product version range it was tested with
`brand`	`product` (name in the top bar), `product_native`, `tagline`, `owner`, `copyright`, `logo` (light/dark files), `favicon`, `about` page text, `support` contact
`theme`	the theme token set (`finance-navy`), overrides of tokens (`--ac`, `--bg`...), default mode (dark/light/system), density
`modules`	each module id with `version` range, `required` (Health degraded if refused) and `enabled` default
`lenses`	which lenses are offered, the default lens, per-lens title overrides
`home`	the default landing per lens, cards and chips order, the welcome text
`menus`	top-level menu order; menu entries hidden for the whole product
`features`	presets of switches (today's `config/lenses/features.yaml`)
`docs`	which Help sections and tutorials show, their order, the docs title ("Drishti Finance Help"), extra pages from the distribution's folder
`bindings.defaults`	default bindings for this product (section 5.5)
`demo`	whether the demo lake ships and is on
`glossary`	extra terms
`locale`	default language and formats

6.3 Examples

- examples/distributions/drishti.finance.yaml: banking + capital markets
- economy; navy theme; Risk lens default; Help in the banking order; risklake defaults.
- examples/distributions/drishti.genomics.yaml: a hypothetical `genomics` module + the kernel; teal theme; Cohorts lens; Help with the kernel's generic guides and the genomics module's; no banking word anywhere.

6.4 How Help and docs follow

Help is assembled at start from the kernel's pages plus each loaded module's pages, filtered by the distribution's `docs` (sections and order), then by the person's lens and functions (as today). The product name in every page is the brand's (`{product}`), never a literal (the branding rule: the 26-literal fix becomes a P3 gate). Exported PDFs, the About page and the console title use the brand. Docs built for print (`docs build --distribution drishti.finance`) render the same selection.

6.5 What a distribution may not do

It cannot add code, connectors or credentials; it names modules, never contains them. It cannot widen a licence (the licence still decides). It cannot rename functions (command words are the modules').

6.6 Distribution builds

`drishti.py dist build drishti.finance.yaml` (P3) makes `drishti-finance-<version>.zip`: the kernel jar, the console, the connector plugins, the named `.dmod` files (unchanged, signed by their publishers), the distribution file and the brand's assets. Installing it is unzipping it; the modules are installed into `modules.path` on first start (approved by the installer's act, audited).

---

7. Lifecycle and tooling

7.1 The command line

Command	Does
`drishti.py module new <id> --domain <d> [--template content|compute|full]`	a new module Maven project from the SDK archetype (section 8.3)
`drishti.py module build [--sign-with dev|<key>]`	builds the jars (Maven), runs the module's tests against the SDK harness, checks (no secrets, no kernel internals, docs gates, contracts), writes `target/dmod/<id>-<version>.dmod`
`drishti.py module sign <file> --key <publisher key>`	signs (or re-signs) a `.dmod` with a publisher key (an HSM or a key file; never stored in the repo)
`drishti.py module verify <file> [--deep]`	V1–V7 against a running or offline kernel; `--deep` runs its conformance tests
`drishti.py module deploy <file> --placement <file> --plan / --apply <plan>`	the scriptable deployment: plan, apply, status, approve, cancel, rollback (section 7.6); `install` below is its short form
`drishti.py module install <file> [--place <part>=<roles or nodes> …]`	uploads to the inbox and proposes the install with its placement (four eyes); with approval: copies to the placed nodes only, stages, activates them together, writes `active.yaml`; *restart needed* on those nodes for code (section 4.11)
`drishti.py module list [--json]`	`lsmod`: id, version, state, licence, taint, needs bound/unbound
`drishti.py module info <id|file>`	`modinfo`: the manifest, signature, publisher, contributions, needs
`drishti.py module run [--shots] [--two-people]`	the dev loop: the kernel + this module only, sample data, hot reload (section 8.4)
`drishti.py module remove <id> [--version v]`	proposes removal (four eyes) with a removal report (what disappears, what is kept); refused while another active module requires it; section 10
`drishti.py module purge-state <id>` / `prune`	deletes a removed module's state folder (four eyes) / inactive versions' files
`drishti.py module upgrade <file>` / `rollback <id>`	install a newer version beside the old; switch `active.yaml`; rollback switches back (both versions stay until `prune`)
`drishti.py bindings list | set | migrate | check`	section 5
`drishti.py dist build | check <file>`	section 6

Transcripts: module new (docs/architecture/kernel-and-modules/examples/cli/module-new.txt), module build (docs/architecture/kernel-and-modules/examples/cli/module-build.txt), module install (docs/architecture/kernel-and-modules/examples/cli/module-install.txt), module list (docs/architecture/kernel-and-modules/examples/cli/module-list.txt), module remove (docs/architecture/kernel-and-modules/examples/cli/module-remove.txt).

7.2 Admin → Modules

The screen of mock-up (a): installed modules with version, publisher, state, licence, needs, taint; refused ones with the reason and the fix; *Install module…* (upload `.dmod` → verify report → propose); *Remove*, *Upgrade*, *Roll back*, *Switch off*; a pending-approvals strip (four eyes); *restart needed* banner with the nodes affected.

7.3 Four-eyes install

Install, upgrade, rollback, remove and every binding change are **proposals** in the Approvals inbox (the kernel's one approvals engine): the proposer cannot approve; the approver sees the verify report (signature, publisher, kernel range, dependencies, licence, needs and which would be unbound, resource budget, what it contributes: functions, APIs, lenses, Help) and the effect (*adds 60 functions, 6 lenses; needs 3 bindings you have not made*). Approved → applied at the next (rolling) restart in P2.

7.4 Upgrade and rollback

- Versions are installed side by side (`banking/3.0.0`, `banking/3.0.1`); `active.yaml` picks one.
- Upgrade: verify the new one with the current set (dependents' ranges still met), approve, restart (rolling), `module.started` with the new version; the module's `ModuleMigration` (P4) upgrades its own state files.
- Rollback: switch `active.yaml` back, restart; state written by the newer version must be readable by the older or the module declares `rollback: false` for that pair (then rollback is refused with words).
- `module prune` deletes inactive versions after a retention (default: keep one previous).

7.5 Health and taint

Taint (Linux's word, the owner's frame) marks an instance that runs something unsupported; it does not stop anything:

Taint	Cause
`D` development-signed	a module signed with the SDK's development key (allowed only when `drishti.modules.allow-dev: true`)
`O` override	an administrator forced a module past a soft check (resource budget)
`R` out-of-range	`drishti.modules.ignore-kernel-range: true` (development only)
`U` unknown publisher	a publisher added to trust by the site, not Drishti's chain

Health shows *TAINTED (D: banking-dev 3.1.0-SNAPSHOT)*; support bundles carry it.

7.6 Deployment: one API, a script and an Admin page

Deploying a module onto a cluster (install, upgrade, move a part, undeploy, roll back) is possible **from a Python script and from an Admin page**, and both are thin clients of **one server API**: one way, no second implementation.

The deployment API (kernel, `/api/v1/admin/modules/deployments`, administrators only, audited):

Call	Does
`POST …/plans`	takes a `.dmod` (uploaded to the module inbox, or one already in the store folder) and a placement (section 4.11); verifies signature, licence and kernel range; computes **the plan**: which parts go on which nodes, memory headroom per node, singleton leaders, what is replaced or removed, warnings (heavy parts, heavy-on-all refused without `allow-all`); **changes nothing**; returns the plan with its id and the **cluster fingerprint** it was computed against
`POST … ({plan})`	proposes applying a plan: a deployment in state `PROPOSED`, waiting for a second administrator (four eyes)
`POST …/<id>/approve`	the approval (never by the proposer)
`POST …/<id>/apply`	executes **that plan only**: refused with `DRS-1029` *the cluster changed since the plan* when the fingerprint, placement or module set differs; then copies to placed nodes, stages, verifies, activates together (section 4.11.4)
`GET …/<id>` (and a live channel topic)	status per node and step: `copying`, `staged`, `verified`, `activated`, `failed (reason)`
`POST …/<id>/cancel`	before activation: aborts on every placed node, staged files removed
`POST …/<id>/rollback`	after activation: the previous active version and placement, as a new plan of its own (four eyes)
`GET …`	the audit list: who proposed, who approved, the plan, the outcome, timings

Undeploy and *move a part* are plans too (placement without the module, or with a part's nodes changed); the same calls apply them.

The Python script: `drishti.py module deploy` (part of the existing CLI, in `tools/modulecli.py`), Terraform-style:

```
drishti.py module deploy banking-3.0.1.dmod --placement config/placement.yaml --plan [--out plan.json] [--json]
drishti.py module deploy --apply plan.json           # this plan only; non-zero exit on failure or a stale plan
drishti.py module deploy status <deployment-id> [--watch] [--json]
drishti.py module deploy approve <deployment-id> --reason "..."   # the second administrator
drishti.py module deploy cancel | rollback <deployment-id>
drishti.py module undeploy banking --plan               # a plan that removes it (section 10)
```

- `--plan` prints exactly what goes where, writes the plan file (the API's plan JSON, with its id and fingerprint), and changes nothing.
- `--apply <plan>` proposes the plan if it is not yet proposed, waits for the approval (`--wait-approval`, default 0: return at once with exit 3 *waiting for approval*), applies it, prints progress per node, and returns **0** on success, **1** on failure, **2** on a stale plan, **3** while waiting for approval: scriptable in CI and schedulers.
- **Auth**: an administrator's API token from `DRISHTI_TOKEN` (or the keyring the CLI already uses), **never on the command line**; the server is `--server` or `DRISHTI_SERVER`.
- **Wrappers for schedulers and CI**: `tools/deploy-module.sh` and `tools/deploy-module.ps1`, a few lines each, calling `drishti.py module deploy` with the same arguments and exit codes (the project's convention for scripts people schedule: a `.sh` and a `.ps1` beside the Python). No separate Python deploy script: one implementation (question Q20).
- The full transcript for banking on the 3-node cluster: cli/module-deploy.txt; the plan file: site/deployment-plan.json.

The Admin page: Admin → Modules → Deploy, a guided flow built only from Drishti Elements (the no-bespoke-UI rule):

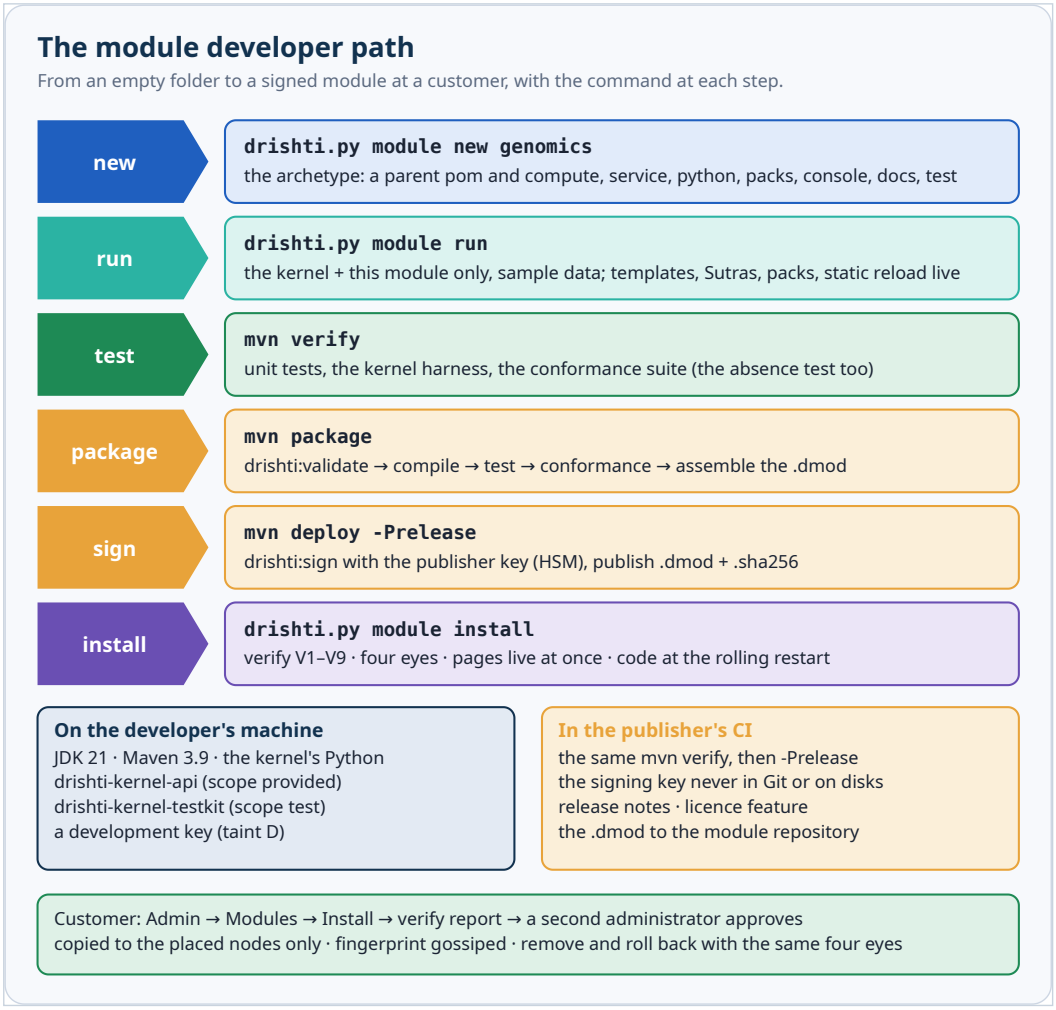
1. **Choose the `.dmod`**: upload, or pick from the module store folder; the signature, publisher, licence feature and kernel range are verified and shown.
2. **Placement**: per part, nodes or roles from a **node picker** that shows each node's role, free memory in the module budget and current modules; heavy parts warn as they widen; heavy-on-all needs the explicit *allow all* confirmation.
3. **The plan**: the same plan the CLI prints, from the same API call: the node × part matrix before and after, headroom, singletons, warnings.
4. **Approval**: proposed into the Approvals inbox; a second administrator approves (the page shows who must act).
5. **Apply** with live progress per node (copy, stage, verify, activate) over the live channel, with *Cancel* before activation and *Roll back* after.
6. **Result**: links to Health, the node × module matrix (Admin → Cluster) and the deployment's audit entry.

The same page offers **Upgrade** (a newer `.dmod`, placement carried over), **Move a part** (placement only) and **Undeploy**, each through the same plan, approve, apply steps. **Deployments** (a tab beside it) lists every deployment: who, what, the plan, the approval, the outcome. Mock-ups: section 15 (e).

Tests: the CLI and the page produce **the identical plan** (byte-equal JSON) for the same `.dmod`, placement and cluster; `--apply` of a plan computed before a node joined, a placement change or another deployment is refused (exit 2, `DRS-1029`), from both front ends; four eyes holds on both (the proposer's approve is refused, from the CLI and the page); a failed stage on one node aborts on all and exits 1.

8. The Module SDK and the developer path

A module is **its own Maven project**, built, tested and signed apart from the kernel, by anyone who has the SDK: a Drishti team, a partner or a customer's own developers. This section is the path from an empty folder to a signed module running at a customer, with the command at every step.



8.1 What the SDK contains

Part	Contents
`drishti-kernel-api-<v>.jar`	the SPIs and service interfaces of section 2.3, Javadoc; Maven `com.ash.drishti.sdk:drishti-kernel-api`, always scope **`provided`** (the kernel supplies it at run time; never inside a `.dmod`)
`drishti-kernel-testkit-<v>.jar`	the harness (8.5), `ModuleConformanceSuite` (8.6) and `AbsenceSuite` (10.11); scope `test`
`drishti-module-parent` (pom)	the parent every module project inherits: Java 21, the versions of the shared libraries (section 4.4), the plugin's defaults, the licence-header check
`drishti-module-maven-plugin`	the goals `validate`, `conformance`, `assemble`, `sign`, `publish` (8.2)
`drishti-module-archetype`	the Maven archetype `module new` wraps (8.3)
`@drishti/elements-*` `.d.ts`	the Elements core typings (from ELEMENTS.md's distribution channels), the plugin.yaml format 2 schema, the kernel templates' macro reference
`drishti_kernel` Python	the compute SDK's types and a local sandbox runner for tests; the console page-handler types
schemas	`module.schema.json`, `needs.schema.json`, `contract.schema.json`, `plugin.schema.json` (format 2), `distribution.schema.json` (editors validate as you type)
`drishti.py module …`	the command line of section 7.1 plus `new` and `run` (8.3, 8.4), for teams without Maven expertise
a sample	the `genomics` sample module (examples in `examples/genomics/`), used by the SDK's own tests and by `drishti.genomics`

All of it is published to the site's (or Drishti's partner) Maven repository; nothing needs the kernel's sources.

8.2 The module as a Maven project

One **multi-module Maven project** per module. The parent pom inherits `drishti-module-parent`, depends on `drishti-kernel-api` with scope `provided`, lists the sub-modules and runs the `drishti-module-maven-plugin` once, after them. Each sub-module maps to one part of the `.dmod` (section 3.2):

Sub-module	Holds	Into the `.dmod`	Needed when
`<id>-compute`	Java compute: engines, `KindProvider`s, pivot measures, functions	`java/`	the module computes in Java
`<id>-service`	the `DrishtiModule` entry point, `ModuleSettings`, `@ModuleApi` controllers, the other SPIs	`java/`	it has an API or any Java
`<id>-python`	Python compute items and packs' Python; `requirements.txt` inside the kernel's lock; pytest	`python/`	it has Python
`<id>-packs`	packs (kinds, contracts, Sutras, samples), `data/needs.yaml`, lenses, `functions.yaml`, BI, feeds	`packs/`, `data/`, `lenses/`, `bi/`, `feeds/`	always (a module without kinds or Sutras has nothing to show)
`<id>-console`	`plugin.yaml` (format 2), its own templates, static files, i18n, page handlers, the domain Elements library	`console/`, `elements/`	it has pages beyond Sutras (section 9)
`<id>-docs`	README, Help, the guide, the tutorial, glossary, the docs register rows, images	`docs/`	always (the docs rule: one guide + one tutorial per feature)
`<id>-test`	harness tests, the conformance and absence suites, two-people tests	never packed	always

The content-only template (economy's shape) has only `-packs`, `-docs` and `-test`; Maven builds what exists.

The plugin's goals, bound to Maven's lifecycle so the ordinary commands do the work:

Phase	Goal	Does
`validate`	`drishti:validate`	`module.yaml` against `module.schema.json`; id and version equal the pom's; `needs.yaml`, contracts, Sutras (Rachana 1), `plugin.yaml` format 2, settings schema parse; every route, word and menu id unique; no connector or secret keys (section 5.10)
`compile`, `test`	Maven's own	the jars; unit tests; pytest through the `-python` sub-module
`verify`	`drishti:conformance`	runs `<id>-test` against the kernel harness, then the conformance checks of 8.6 that need no kernel (lints, licence headers, CSP, SRI)
`package`	`drishti:assemble`, `drishti:sign`	lays the files into the `.dmod` layout, writes `DMOD.MF` (SHA-256 of every entry), signs with the configured key (`dev` by default: taint `D`), writes `target/dmod/<id>-<version>.dmod` and its `.sha256`
`deploy` (`-Prelease`)	`drishti:publish`	re-signs with the publisher key (PKCS#11 URI, an HSM in CI), publishes the `.dmod`, its `.sha256` and release notes to the module repository

The complete example: genomics/pom.xml and its project tree (docs/architecture/kernel-and-modules/examples/genomics/project-tree.txt).

8.3 Creating a module: the archetype and `module new`

Two doors, one result:

```
$ mvn archetype:generate -DarchetypeGroupId=com.ash.drishti.sdk -DarchetypeArtifactId=drishti-module-archetype \
      -DarchetypeVersion=1.0.0 -DgroupId=com.example.genomics -DartifactId=genomics -Dtemplate=full -Ddomain=genomics
$ python3 tools/drishti.py module new genomics --domain genomics --template full --publisher example-labs
```

`module new` **wraps the archetype** (it runs the bundled Maven wrapper `mvnw`, so a team without Maven on the path or Maven knowledge still gets the same tree) and then asks the three things a team usually does not know: the publisher id, the licence feature, and the first kind's name. Templates: `content` (packs, Sutras, lenses, docs), `compute` (+ Java compute and Python), `full` (+ console pages and a domain Elements library). The generated project builds green on the first `mvn verify` (its sample kind, Sutra, page, guide, tutorial and tests are complete, not stubs). The transcript: module new (docs/architecture/kernel-and-modules/examples/cli/module-new.txt).

8.4 The dev loop: `module run`

```
$ cd genomics && python3 tools/drishti.py module run           # or: mvn drishti:run
```

- Starts the **kernel thin jar and its console with this module only** (and its `requires`, fetched from the module repository), a data folder under `target/run/`, the demo lake built from the module's `samples/`, a development licence naming its feature, the development key (taint `D`), on free ports printed at start (never the usual 18480/17480).
- **Hot reload** while it runs: templates, `plugin.yaml` (the live route table, section 9.8), static files, i18n, Sutras, packs, contracts, lenses, Help and guides reload on save (the kernel's pack reloader plus a watcher on `target/run/modules/<id>/`, which is a link to the sub-modules' source folders). Java and console Python handlers: `module run` recompiles and restarts the module's part only (a few seconds; the kernel stays up, the harness's hot unload of section 4.4 is used in development before P5 uses it in production).
- `--shots` takes the docs' screenshots of the module's pages into `<id>-docs/img/` (the same driver as `tools/docs/screenshots.py`).
- `--two-people` opens two sessions as two test people (the session-isolation check by eye).
- Nothing it writes leaves `target/`; Ctrl-C removes its temporary files.

8.5 The test harness (kernel + one module)

- `KernelHarness.with(module("target/dmod")).demoLake().start()` boots the **real kernel** (thin jar, in process) with only this module (and its `requires`, fetched from the module repository), a temporary data folder (`@TempDir`), the demo lake from the module's samples, a development licence and key (taint `D`).
- Gives: a REST client as a test person, the `Kinds` API with fixture bindings, a console client (pages, the route table, search, Help), a headless browser page (Playwright) for composed pages, a Python pool, the module-off and module-removed variants, and the suites of 8.6 and 10.11.
- Python side: `pytest` plugin `drishti_kernel.testing` (the console harness with the module mounted).

8.6 The conformance suite

Every module passes it to be signed with a publisher key (`drishti:sign` refuses a release build that has not passed it); `module verify --deep` runs it again at the customer.

#	Check	How
C1	**No kernel internals**	ArchUnit: no class imports `com.ash.drishti..` outside `kernel.api`; the jar's classes resolve against the API loader alone; console Python imports only `drishti_kernel` and the module's own package
C2	**No bespoke UI**	templates use only `kernel/*` layouts and `kernel/elements.html` macros plus the module's own Elements library; no `<table>`, `<canvas>`, `<svg>` drawing or hand-made widgets outside Elements components; JS imports only `@drishti/elements-*` public entry points
C3	**Theme tokens only**	CSS lint: colours, spacing, fonts and radii are `var(--…)` tokens; no hex, rgb or named colours; no `!important` over kernel classes
C4	**`drishti_` asset prefix**	every file under `static/` and `elements/` is `drishti_<name>` (the static-asset rule); vendor libraries are not allowed in a module (they belong to the kernel's vendored set)
C5	**CSP-clean**	no inline `<script>`, no `on*=` attributes, no `javascript:` URLs, no `eval`/`new Function`, no remote origins; SRI hashes for every static file written into the manifest
C6	**Routes namespaced**	every route under `/m/<id>/`; aliases and words checked against the reserved names (9.9); no route registered by code (no FastAPI router, no `@app.get`)
C7	**Help, guide, tutorial present**	`docs/README.md` with front matter; a Help section; one guide and one tutorial per `docs.features` row; the docs gates (links, images, register) on the module's `docs/`
C8	**Data contracts present**	every kind in `provides.kinds` has a contract (`contract-format: 1`) with `binding-hint:` and no `storage:`; every need names a contract range
C9	**No secrets, no connectors**	the secret patterns and connector keys of section 5.10 in every file, including templates and samples
C10	**Licence headers**	the publisher's header on every source file (the parent pom's check)
C11	**Functions answer**	every word in `functions.yaml` answers through the harness (a page, a view or an API)
C12	**Two people at once**	the session-isolation test: two people, different row rules, concurrent requests, no crossed data in any page, API or cache
C13	**Off means gone**	the module switched off: every surface refused (the existing guarantee, ACCESS_MODEL section 6)
C14	**Absent everywhere**	the absence suite of section 10.11: install, record every surface, remove, assert each one gone and nothing left in the registry
C15	**Class loader unloads**	50 load/unload cycles in the harness; the loader is collected (section 4.4)

8.7 The release path

1. **Version**: semver in the pom and `module.yaml` (validated equal); release notes from the module's CHANGELOG.
2. **Build in CI**: `mvn -Prelease deploy` on the publisher's CI: `verify` (tests, harness, conformance), `package`, `sign` with the **publisher key** (an HSM through PKCS#11; the key never on disk or in Git), `publish` to the module repository with its `.sha256`.
3. **Certificate**: the publisher's certificate is issued under the Drishti module root (section 11.1, question Q7); a partner signs its own modules with its own certificate; Drishti signs banking, economy and capital markets.
4. **Deliver**: the customer receives the `.dmod` (download from the module repository, or a distribution build of section 6.6).
5. **Install at the customer**: Admin → Modules → *Install module…* or `module install` (section 7.1): verify report (V1–V9), four eyes, active on approval (pages at once for content and templates; code at the rolling restart in P2: section 9.8).
6. **Support**: `module info` and the support bundle carry the module's id, version, publisher, signature fingerprint and taint.

8.8 From start to delivery: the steps

Step	Who	Command	Result
1 Get the SDK	developer	add the module repository to `~/.m2/settings.xml` (or unpack the SDK zip)	`drishti-module-parent`, the API, the testkit, the archetype
2 Create	developer	`drishti.py module new genomics --template full` (or `mvn archetype:generate …`)	a project that builds green
3 Model the data	developer	edit `<id>-packs`: contracts, `needs.yaml`, samples	kinds the kernel can bind (section 5)
4 Show it	developer	Sutras first; composed pages in `<id>-console` (section 9)	pages, words, menus, Home cards
5 Compute	developer	Java in `<id>-compute`/`-service`, Python in `<id>-python`	engines, `KindProvider`s, APIs
6 Run	developer	`drishti.py module run`	the kernel + this module, hot reload
7 Document	developer	`<id>-docs`: guide, tutorial, Help, glossary; `module run --shots`	the docs gates pass
8 Test	developer, CI	`mvn verify`	unit, harness, conformance (C1–C15), absence
9 Package	CI	`mvn package`	`target/dmod/genomics-0.1.0.dmod` (development-signed)
10 Sign and publish	publisher's CI	`mvn -Prelease deploy`	signed `.dmod` + `.sha256` in the module repository
11 Install	customer administrators	`drishti.py module install genomics-0.1.0.dmod` + approval	verify report, four eyes, LOADED
12 Bind data	customer data administrator	Admin → Data bindings (or `drishti.py bindings set …`)	needs `BOUND` (section 5)
13 Upgrade or remove	customer administrators	`module upgrade` / `rollback` / `remove`	section 7.4 and section 10

8.9 Worked example: the genomics sample module

The files, all in `examples/genomics/` (laid out as inside the `.dmod`; the project tree shows the source sub-modules):

File	Shows
project-tree.txt	the Maven project and where each folder lands in the `.dmod`
pom.xml	the parent pom: `provided` kernel API, seven sub-modules, the plugin's goals, the release profile
module.yaml	the manifest of a partner-published module with Java, Python, a computed kind and pages
console/plugin.yaml	format 2: routes, API proxy, words, menus, Home cards, slots, settings, search
console/templates/variant_explorer.html	a composed page (section 9.11)
packs/genomics/sutras/variant-call.v1.sutra.yaml	a Sutra (tier 1)

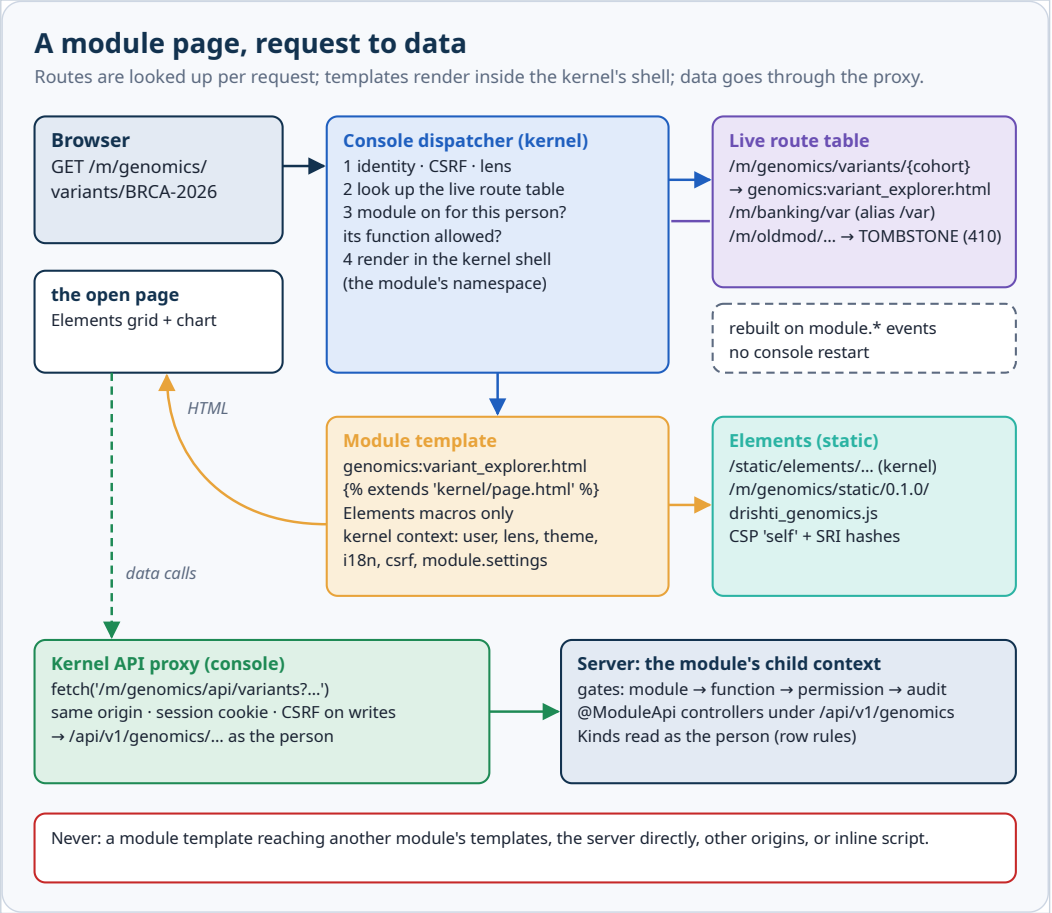
8.10 Banking's equivalent

Banking becomes the same shape at `modules/banking/` in this repository (Q9): `banking-compute` ← `drishti-compute-banking`, `banking-service` ← `drishti-banking`, `banking-python` ← `drishti_banking` and the packs' Python, `banking-packs` ← `config/packs/banking/` (+ its lenses, BI and lake forms), `banking-console` ← `drishti-console/plugins/banking/` (plugin.yaml format 1 → 2, the nine route files reduced to context builders, the ~35 API pass-throughs replaced by the kernel's proxy), `banking-docs` ← the seven banking guides and nine tutorials, `banking-test` ← the banking tests, the VaR exact check and `BankingAbsenceTest`. The file-by-file mapping: banking/project-mapping.txt; the format-2 plugin file: banking/console/plugin.yaml.

8.11 The developer guide (outline)

A new guide, `docs/guides/MODULE_DEVELOPER_GUIDE.md` (P2; one guide + one tutorial, per the docs rule): 1. What a module is; the kernel API and its versions. 2. The Maven project, `module new`, the archetype. 3. Data: kinds, contracts, needs; testing with the demo lake; never connectors. 4. Java: `DrishtiModule`, the SPIs one by one with examples, REST under your prefix. 5. Python compute items. 6. Pages: Sutras first, composed pages from Elements, your domain library, CSS on tokens, routes and the API proxy. 7. Lenses, functions, menus, Home cards. 8. Help, guides, tutorials, glossary. 9. Settings schema. 10. `module run`, the harness, the conformance and absence suites. 11. Build, sign, publish, install. 12. Upgrades, deprecations, removal and what your users keep. 13. Security rules. 14. Worked examples: genomics, then banking. Tutorial: *Build the genomics sample module from scratch*.

9. Module UI and the developer's own pages



9.1 Three tiers, preferred first

Tier	What the developer writes	Code	When
1 Sutras	YAML pages in Rachana (`packs/<pack>/sutras/*.sutra.yaml`): strips, tables, charts, linked entities, Explain, keys	none	**first choice**: an entity's page, a list, a dashboard of a kind; the kernel draws them, search and Help find them, the designer edits them
2 Composed pages	the module's **own Jinja templates**, built only from Drishti Elements macros and components, rendered inside the kernel's shell; optional Python context builder	a template (+ a small handler)	a screen Sutras cannot express: a workflow, an explorer with linked filters, a playground

Tier	What the developer writes	Code	When
3 A domain Elements library	`elements/<lib>/`: ES modules and custom elements (`<drishti-genomics-variant-track>`) on the Elements core API, CSS on tokens, `.d.ts`, a manifest	JavaScript	a visual the general library lacks (a genome track, a VaR limit bar); used by tiers 1 and 2, never alone

A tier-3 component is offered to Sutras as a panel type and to composed pages as a tag; the no-bespoke-UI rule holds at every tier (conformance C2). Most modules need tier 1 only; banking uses all three.

9.2 Serving a module's own templates: the namespace and the loader

- **One namespace per module**: a module's templates are addressed `<id>:<path>` (`genomics:variant_explorer.html`). The console builds **one Jinja environment per module** (`ImmutableSandboxedEnvironment`, autoescape on, the kernel's filters and globals only) whose loader is a `PrefixLoader` with exactly two prefixes: `kernel` → the kernel's **public** templates (`kernel/page.html`, `kernel/elements.html`, `kernel/partials/*`), and `<id>` → the module's `console/templates/`. Nothing else is reachable: not the kernel's private templates (`terminal/*`, `admin/*`), not another module's namespace.
- The kernel's own pages keep their environment; it has **no** module loader at all, so a kernel template cannot include a module's (principle 1, structurally rather than by review).
- Bytecode cache per module environment (`console-cache/templates/<id>/<version>/`); dropping the environment drops its cache.
- Today's `templates:` list (format 1) and the shared loader chain (`Jinja2Templates(directory=[web, plugin dirs…])`, where a module's file could shadow a kernel file of the same name) are **removed** with format 2.

9.3 Layouts, Elements macros and extension slots

- A composed page **extends** `kernel/page.html` (the shell: top bar, menus, command line, notices, the live channel, the theme) and fills its blocks: `title`, `assets`, `page`, optional `aside` and `keys`. `kernel/page.html` is part of the kernel API (`plugin-api` version; its blocks change only with a MAJOR).
- It uses only the macros of `kernel/elements.html` (page header, KPI strip, filter bar, grid, chart, map, tabs, panel, form fields, notices, Explain button, export) and tags of Elements libraries (kernel or its own). Each macro emits the markup of an Elements component with its data source; the JavaScript is the kernel's.
- **No override of kernel templates.** A module that ships `terminal/_base.html` or any `kernel/…` path is refused at V6 (*claims a kernel template*). The only ways into a kernel page are **declared extension slots**:

Slot	Kernel page	Today's equivalent
`home.landing`, `home.cards`	Home, per lens	landings (`trading_landing`), lens cards
`pivot.top`, `pivot.below`	Pivot, by lens or kind	`screens.pivot` (the MR lens panes)
`entity.panels`	an entity's view, by kind	view augmenters' panels
`admin.tabs`	Admin, a tab	`/admin/scopes`
`help.section`	Help centre	`help.yaml` heads
`explain.drawer`	the Explain drawer, by measure	`Explainer` SPI's text

A slot template is in the module's namespace, gets the slot's context only, and is drawn in an isolated container (`<section data-module="banking">`) so its removal leaves the page whole.

9.4 The template context the kernel provides

Name	What	Not included
`user`	id, display name, roles, locale	tokens, cookies, the session id, other people
`lens`	the person's current lens id and title	—
`theme`	mode (dark/light), density, token set name	raw colours (CSS uses tokens)
`t(key, **args)`	i18n: the module's bundle, then the kernel's	—
`csrf_token`	for forms posting through the proxy	—
`module`	`id`, `version`, `title`, `settings` (only keys listed in `settings.expose-to-pages`)	secrets (there are none), other settings
`route`	the route's id, typed params, title	the raw request
`url_for(route_id, **params)`	a link to the module's own route (canonical address) or `kernel:<page>`	another module's routes (use command words or `link()` to entities)
`api(path, **query)`	the proxied URL of one of its declared API endpoints (9.5)	the server's address
`asset(path)`, `elements(lib)`	versioned `<link>`/`<script type=module>` tags with SRI (9.7)	other modules' assets
`product`, `business_date`, `now`	the brand (distribution), the business date	—
the handler's dict	whatever the module's optional handler returns (plain data, JSON-serialisable)	objects, callables

A **handler** (`handler: variant_explorer` → `drishti_modules.genomics.pages.variant_explorer(ctx)`) is a plain function given a read-only page context (the person, the params, a `client` bound to the module's API as that person) that returns a dict. It cannot register routes, read the request, set cookies or reach module-global state: per-person data lives only in the call (session isolation).

9.5 How a page calls its module's API: the kernel's API proxy

- The browser calls `/m/<id>/api/<path>` on the console's own origin; the console's proxy forwards to the server's `/api/v1/<prefix>/<path>` **as the person** (the console's existing backend client and the person's session), adds the request id and `Drishti-Module: <id>`, and streams the answer back (JSON, JSON Lines, Arrow, xlsx keep their media types).

- **Only declared endpoints** pass: `api.proxy` in `plugin.yaml` lists method + path patterns (+ an optional permission); anything else is 404 at the console. Non-GET methods need the CSRF token. Timeouts and size limits come from `api.limits`.
- The server applies its gates in order (module → function → permission → audit, section 4.5); the proxy adds nothing it could bypass.
- The proxy keeps **no per-person state**: no cache unless keyed by person and module; it is the same code for every module (session-isolation rule).
- This replaces today's hand-written pass-throughs (banking's ~35 `/api/var/**`, `/api/exposure/**` ... handlers); their console paths stay as `api.aliases` for one release.
- Elements data sources (`source=api("/variants")`) and the module's own JS (`fetch(api.url("/variants"))` from the Elements data library) both use it; the JS never builds URLs to other paths (lint C5, CSP `connect-src 'self'`).

9.6 Template sandboxing: what is forbidden

Forbidden	Enforced by
Inline `<script>`, `on*=` attributes, `javascript:` URLs, `eval`	CSP `script-src 'self'` (nonce only for the kernel shell), conformance C5
Raw HTML from data (the `safe` filter, `Markup`)	the module environment has no `safe` filter and no `Markup` global; autoescape forced; only Elements macros emit markup
Styles other than tokens; `style=` with colours	conformance C3; CSP `style-src 'self'`
Network calls to other origins; `<iframe>`, remote images, fonts or scripts	CSP `connect-src 'self'`, `frame-src 'none'`, `img-src 'self' data:`; lint C5
`fetch` to anything but `/m/<id>/api/…`	lint C5 (CSP cannot restrict paths), proxy 404 for undeclared endpoints
Other modules' templates, assets, JS modules or globals	the per-module loader (9.2); the Elements loader refuses another module's library; lint C1
Kernel internals: private templates, `request`, `app.state`, the backend client, Python attributes starting with `_`	the sandboxed environment; the context of 9.4 is all there is
Forms posting outside the proxy	lint; the kernel's CSRF check on every non-GET

9.7 Static assets

- Served at `/m/<id>/static/<version>/<path>` from the module's `console/static/` (and `elements/<lib>/` at `/m/<id>/elements/<lib>/<version>/`), read-only, `Cache-Control: public, max-age=31536000, immutable`: the version in the path **busts the cache** on upgrade; rollback serves the older version's path again.
- **SRI**: `drishti:assemble` writes the SHA-384 of every static file into `console/assets.lock`; `asset()` emits `integrity=` from it; the console refuses to serve a file whose hash differs (tamper check at run time, besides V2 at load).
- **CSP**: `script-src 'self'` and `style-src 'self'`; no inline, no remote origins; the module cannot change the policy.
- `drishti_` **prefix** on every file (C4); vendor libraries are the kernel's only.
- Served only to people for whom the module is on and whose lens uses it (today's `_module_asset_refused`, unchanged); a removed module's paths answer 410 (section 10.10).

9.8 Routes: declared, namespaced, live

- **Declared, never registered by code.** Every page and every API pass-through is an entry of `plugin.yaml` (`routes:`, `api.proxy:`). No module code runs at import to add a route (format 1's `routers:` and FastAPI `include_router` are removed). A route names what draws it: `sutra:` (tier 1), `page:` (tier 2, with an optional `handler:`), or `moved:` (a 308 to another route for one release).
- **The namespace**: every route lives under `/m/<id>/…`; ids are unique in the module and are what links, menus, cards and words use (`route: variant-explorer`), never raw paths.
- **The live route table.** The console registers its own pages at start as today plus **one dispatcher** for `/m/{id}/{path}` and one fall-through for declared aliases (consulted only after the kernel's routes miss). The dispatcher looks the path up **per request** in an immutable **route table snapshot**; a new snapshot is built from the loaded modules' `plugin.yaml` files and swapped atomically when a `module.*` event or a cluster config version arrives (copy-on-write; no lock on the request path; nothing per person in it). Each entry carries the module, function, permission and state, so the dispatcher answers: render, *not available* (404 for an off module, as today), or **tombstone** (410 for a removed one, section 10.10).
- **What needs no console restart in P2**: switching a module off or on; **removing** a module (every page, word, menu, asset gone at once); installing, upgrading or rolling back a module whose console part is **declarative** (Sutras, templates, static, menus, words, Help: no `handlers:`), once the server reports it loaded.
- **What still needs a restart in P2**: a module's **console Python handlers** newly installed or changed (the console imports module Python only at start until P5, so no two versions of a module's Python ever share a process), and the server's Java and Python code (section 4.10). Until then the new routes are listed to administrators as *installed, restart pending* and absent for everyone else. P5 makes both hot (with the leak check).
- **Deep links and bookmarks.** The canonical address is `/m/<id>/<path>`; an alias answers the same page (200) with `<link rel="canonical">`; page state lives in the query string (Elements' URL state), so a link reproduces the view. A path that changes between versions is declared `moved:` (308 for one release). Upgrades keep route ids and paths unless a `moved:` says otherwise. After removal, a bookmark lands on the removed-module page (10.10), and Drishti's own bookmark list marks it *needs module X*.

9.9 Aliases, short words and reserved names

- A module may declare **short path aliases** (`alias: /variants`, banking's `/var`) and **command words**. Both are checked at V6 against the **reserved names**: the kernel's paths (`/t`, `/pivot`, `/bi`, `/build`, `/admin`, `/help`, `/data`, `/static`, `/m`, `/api`, `/embed`, `/public`, `/healthz`, `/readyz`, ...), the kernel's words (`PIVOT`, `BI`, `FUNC`, `FORM`, `HELP`, `WHAT`, `DATAMAP`, `EXPORT`, ...), the site's own list (`config/reserved-names.yaml`), and every name already claimed by an installed module.
- **Conflicts are refused at load with the reason**, never resolved by order: *genomics 0.1.0 refused: alias /variants is claimed by biobank 2.1.0 (route biobank:variants)*. The first-installed keeps it; the newcomer is refused whole (refused means absent).
- Path aliases outside `/m/` are allowed for **Drishti-signed modules**; partner modules get them only when the site grants `drishti.modules.<id>.allow-aliases: true` (four eyes) (question Q15). Command words are allowed for all, checked the same way.

9.10 Menus, functions, Home cards and search, declared

`plugin.yaml` declares the module's **menu entries** (merged into the sitemap's menus, tagged `domain: <id>`), its **words** (each to a route, with its grammar), **Home cards and chips** per lens, **slots**, the **settings page** (generated from the settings schema), and extra **search** entries. The kernel builds every door from these declarations plus the module's Sutras, functions and Help; nothing is drawn from module code. That is what makes

removal complete: withdrawing the declarations withdraws the doors (section 10.3).

9.11 Examples: a genomics variant explorer, banking's VaR explorer

Genomics (plugin.yaml, variant_explorer.html): the route

```
- id: variant-explorer
  path: /variants/{cohort}
  page: variant_explorer.html          # genomics:variant_explorer.html
  handler: variant_explorer            # optional context builder
  function: VARX
  permission: genomics:read
  alias: /variants
```

draws a page that extends `kernel/page.html`, puts a KPI strip (a partial of its own), a filter bar, an **Elements grid** over `api("/variants", cohort=…)` and an **Elements chart** (allele-frequency histogram fed by the grid) side by side, and the module's tier-3 `<drishti-genomics-variant-track>` following the grid's selection. No script in the template; the JS is the kernel's Elements plus `drishti_genomics-explorer.js`.

Banking's VaR explorer today is `terminal/var.html` included through the shared loader, `var_routes.py`'s FastAPI routes and ~10 pass-throughs, and the `drishti_var-*.js` files. As a composed page:

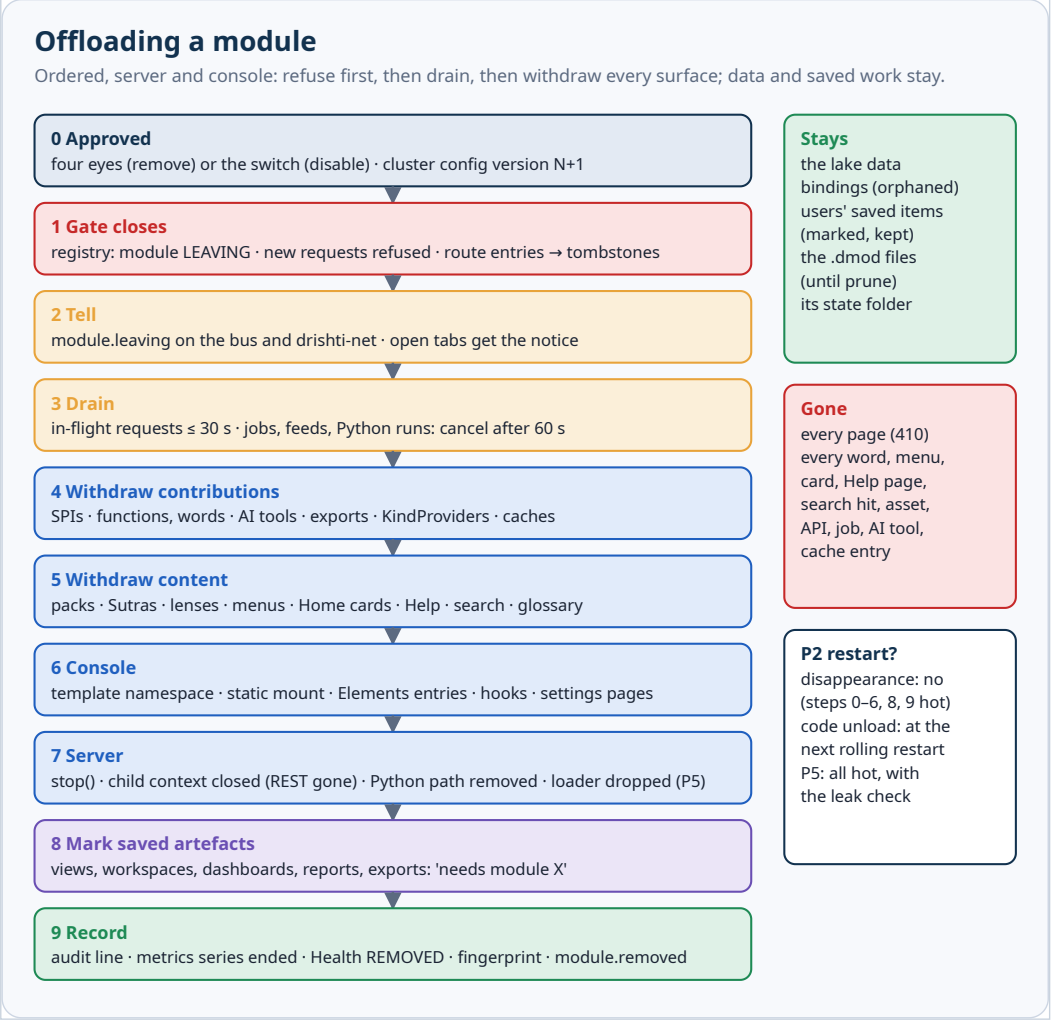
```
- { id: var-explorer, path: "/var/{scope...}", page: var_explorer.html, handler: var.explorer, function: VAR, alias: /var }
```

`banking:var_explorer.html` extends `kernel/page.html`; the handler only parses the scope words (`DESK RATES`) into the API query; the page is a KPI strip (VaR, ES, limit use), an Elements grid of contributions over `api("/var/contributions", scope=…)`, an Elements chart of the P&L distribution, and `drishti-risk`'s `<drishti-risk-limit-bar>` and `<drishti-risk-hierarchy>`; every number is computed on the server (the server-computes-risk rule). `/var` stays as the alias, so bookmarks and typed `VAR …` keep working.

---

10. Offloading a module: everything disappears

The owner's rule: **when a module is removed or disabled, all its pages, and everything else it brought, disappear**, at once, on every node, without breaking an open page, and without losing anything a person saved or any data the site holds.



10.1 Four ways a module leaves

	Disabled (switch off)	Removed	Upgraded	Rolled back
Trigger	`drishti.modules.<id>.enabled: false`, Admin switch (four eyes)	`module remove` (four eyes)	`module upgrade` (four eyes)	`module rollback` (four eyes)
Files	kept	kept until `module prune` (rollback, reinstall)	old and new side by side	both kept
Pages	**404** + notice *not available* (as today)	**410** + notice *no longer installed* (tombstones)	carried over by route id; changed paths 308 (`moved:`)	as upgraded, back
Every other surface	withdrawn (10.3)	withdrawn (10.3)	withdrawn and re-added within the swap	same
Open tabs	notice *switched off*	notice *no longer installed*	notice *updated: reload*	notice *changed: reload*
Saved work	marked *needs module X (off)*	marked *needs module X (not installed)*	untouched, unless the new version drops a function it uses (then marked *needs VARX, removed in 0.2.0*)	same
Schedules, alerts	paused	paused	kept	kept
Data, bindings	stay	stay (bindings orphaned)	stay	stay
Code in the processes (P2)	stays loaded, unreachable	unloaded at the next rolling restart	new code at the rolling restart	same
Reverse	switch on	reinstall (approved): everything marked is restored	rollback	upgrade again

10.2 The unload sequence

Ordered; the same for disable and remove except where marked. Each step is idempotent and logged; a failing step does not stop later steps (it is retried and shown in Health), because **refusing first** (step 1) already makes the module absent to everyone.

#	Step	Server	Console
0	**Approved**	the change (four eyes) writes the cluster configuration version N+1: `genomics: REMOVED` (or `OFF`)	—
1	**Gate closes**	the registry marks the module `LEAVING`: the module gate answers `DRS-1025` (off) or `DRS-1028` (*not installed*) for every new request; the scheduler starts nothing of it	the route-table snapshot is rebuilt with its entries as **off** or **tombstones**; words answer *not available*
2	**Tell**	`module.leaving` on the bus and over `drishti-net`	every open tab of the module gets the event on its live channel (10.4)
3	**Drain**	in-flight requests finish within `drishti.modules.drain-seconds` (30); running jobs, feed runs and Python runs get a cooperative cancel, then a hard cancel after `drishti.modules.cancel-seconds` (60) (10.5)	proxied calls in flight finish or time out with the same limits
4	**Withdraw contributions**	every SPI registration in `ModuleContributions` (engines, `PivotProvider`s, `FunctionProvider`s, view augmenters, explainers, approval sources, sharded measures, Flight forms, capacity and health figures, AI tools, exports, playground templates); `KindProvider`s (its computed kinds become *not available*); its `CacheGroup`s cleared; pivot, BI and Explain cache entries tagged with the module dropped; event subscriptions removed	—
5	**Withdraw content**	packs, kinds, contracts, Sutras, lenses, functions, Home cards, BI content, report definitions, feeds; the search index's entries tagged `module:<id>`; Help pages, guides, tutorials, glossary terms; designer palette entries (its panel types)	menus and the sitemap entries tagged `domain:<id>`; Help centre sections; command-line words, type-ahead and hints (the hooks are keyed by module)
6	**Console**	—	the module's Jinja environment and its bytecode cache dropped; static and Elements mounts withdrawn (paths answer 404/410); Elements manifest entries removed; its settings page removed; P2: its Python handlers stay imported but unreachable (no route points at them) until the restart
7	**Server**	`DrishtiModule.stop()`; the child Spring context closed (its `DispatcherServlet` and every `@ModuleApi` mapping gone); the Python pool's import-path entry removed; P2 removal: the class loader is dropped at the next rolling restart (P5: now, with the leak check); disable keeps it loaded	—
8	**Mark saved work**	the tombstone file `modules/tombstones/<id>.yaml` (its route ids, words, kinds, Sutras, report ids, API prefix) is written; saved artefacts that reference any of them show *needs module X* (10.6); schedules and alerts that need it pause; owners are notified once	—
9	**Record**	audit line (who proposed, who approved, what was withdrawn, counts); the module's metrics series ended (meters removed from the registry; history kept); Health row `REMOVED` / `OFF`; the cluster fingerprint updated; `module.removed` / `module.stopped`	the route table version shown in Admin → Modules

The transcript of a removal with every count: cli/module-remove.txt.

10.3 The inventory: every surface and how it goes

Surface	Contributed by	Removed by (step)	The absence test asserts
Routes and pages	`plugin.yaml routes:`	route table → tombstones (1, 6)	every recorded URL answers 410 with the notice page (404 when off)
Menus	`plugin.yaml menus:`	sitemap entries tagged `domain:<id>` (5)	no menu, Admin tab or BI rail item links into the module, in every lens

Surface	Contributed by	Removed by (step)	The absence test asserts
Lens entries	`lenses/lenses.yaml`	registry lenses withdrawn (5); a person on that lens falls back to the default lens with a notice	the lens list has none of its lenses
Function / command words	`functions.yaml`, `plugin.yaml words:`	registry functions (5); console hooks (5)	each word answers *not available*; `FUNC` lists none
Home cards and chips	`plugin.yaml home:`, `module.yaml home:`	lens cards (5)	Home in every lens has none
Search index entries	Sutras, functions, Help, `search:`	entries tagged `module:<id>` (5)	a search for each of its words, titles and terms finds nothing
Help pages, guides, tutorials	`docs/`, `help.yaml`	Help centre sections, the docs register rows (5)	Help index, Help search, guides list and tutorials list have none; their URLs 410
Glossary	`docs/glossary.yaml`	glossary terms (5)	none of its terms
Static assets	`console/static`, `elements/`	mounts withdrawn (6)	every recorded asset URL answers 410
REST endpoints	`@ModuleApi`	child context closed (7); gate first (1)	every recorded endpoint answers 410 `DRS-1028` (404 `DRS-1025` when off, as today)
SPI registrations	`ServiceLoader`, beans	`ModuleContributions` withdrawn (4)	the registry of contributions has no entry whose class loader is the module's
Scheduled jobs and feeds	`feeds/`, scheduler registrations	suspended, then removed (3, 5)	the scheduler lists none; no run starts after step 1
Python compute	`python.items`, packages	items unregistered (4), import path removed (7)	Build lists none; a run naming one is refused
Caches (pivot, BI, Explain, its own)	`CacheGroup`, tagged entries	cleared (4)	cache reports have no group or entry tagged with it
AI tools	`AiToolProvider`	withdrawn (4)	the assistant's tool list has none
Settings pages	`settings/`, `plugin.yaml settings:`	page removed (6); stored values kept for reinstall	Admin → Modules has no settings tab for it
Sutras	`packs/*/sutras/`	pack withdrawn (5)	no kind matches one of its Sutras; its Sutra URLs 410
Designer palettes	tier-3 panel types	palette entries (5)	the designer offers none
Exports	`ExportProvider`, export buttons	withdrawn (4)	the export list has none; scheduled exports paused
Audit	audit lines carry `module`	**kept** (audit is never removed); the removal itself is audited (9)	the removal's audit line exists
Metrics	meters named `<id>.*`	meters removed, series ended (9)	the metrics endpoint exposes none
Event subscriptions	`events.subscribes`	removed (4)	publishing `data.loaded` reaches no handler of it
Embeds and tokens	embed pages, `tokens/token-scopes.yaml`	scopes withdrawn (5); embed URLs 410	its scopes are not grantable; embeds 410

10.4 Open browser tabs

- The kernel shell keeps one **live channel** per tab (today's `/api/channel`). On `module.leaving` the console sends a `module` event (`{module, state: removed|off|upgraded, title}`) to every channel; the shell (kernel JavaScript, not the module's) handles it.
- On a page **of** that module: the shell freezes the page's Elements (they stop polling and refuse input), shows the kernel's notice *Genomics is no longer installed on this system* with *Go to Home* and, for administrators, the reason and the change id. The page's own JS is not called again; nothing throws (Elements components are told to `detach()` first, and their data sources answer a typed *module gone* result instead of failing requests).
- On a kernel page **with** module parts (a slot, a menu entry, a card, a palette): the shell removes the slot containers (`[data-module=genomics]`), refreshes menus and cards from the next answer, and shows a quiet notice.
- A tab that was asleep reconnects, receives the current route-table version, and does the same; a navigation lands on the 410 page.
- Mock-up: section 15, (d).

10.5 In-flight requests and running jobs

- **Requests**: new ones are refused at step 1; those already past the gate finish within `drain-seconds` (30), then are cancelled (the client gets 503 `DRS-1028` with *the module was removed while this ran*).
- **Jobs, feeds, Python runs**: none starts after step 1; running ones get a cooperative cancel at once, a hard cancel after `cancel-seconds` (60); partial results are discarded, never published (a produced kind is written only on success).
- **Long exports and scheduled reports**: cancelled the same way; their schedules pause (10.6).
- **Four eyes and drain**: the approver sees *9 people have pages open, 2 calls in flight, 0 jobs running* before approving.

10.6 Users' saved artefacts

Saved views, workspaces, dashboards, bookmarks, BI reports, notebooks, scheduled exports, alerts and subscriptions that reference a module's pages, words, kinds, Sutras or functions are **kept, marked and restored, never silently deleted**:

- **How the kernel knows**: the tombstone file (step 8) lists what the module owned; when an artefact is listed or opened, a reference that no longer resolves is looked up there and reported as *needs module genomics*. No artefact is rewritten and **no database change is needed** (no migration; artefacts keep their stored form).
- **Shown**: in lists, the artefact keeps its place with a *needs genomics* chip and is not openable; a workspace or dashboard opens with the missing tiles replaced by a notice tile and everything else working; a BI report shows the visuals that depend on it as notices.
- **Schedules and alerts** pause with status *paused: needs module genomics*; the owner is notified once.

- **Reinstall**: references resolve again, chips disappear, paused items resume (same major version), or stay paused with the list of what is still missing (a new major that dropped something).
- **Deletion** only by the owner (or an administrator acting for them), with the usual confirmation; tombstones are kept `drishti.modules.tombstone-days` (365) (question Q16).

10.7 The data in lakes

Data belongs to the site, bindings to the kernel (section 5): **nothing in a lake is touched**. The module's produced kinds (`var-official`) stay where the site bound them; its kinds' bindings stay in `config/bindings.yaml` and show as *orphaned: no installed module needs this kind* in Admin → Data bindings (never removed automatically); the data stays readable by Pivot, BI and `DATA` through the kernel's connectors only if another module or the kernel defines the kind (otherwise the lake tables are still there for a reinstall). The module's own state folder (`${drishti.data.dir}/module-state/<id>/`) is kept; `module purge-state <id>` (four eyes) deletes it.

10.8 Sessions

Session isolation makes removal clean: nothing module-scoped lives in shared state. Per-person caches keyed by person **and module** are dropped on `module.leaving`; the person's session and cookie are untouched; a person whose current lens was the module's falls back to the distribution's default lens at the next request, with a notice. The absence test (10.11) runs with two people signed in during the removal and asserts neither sees the other's data or a broken page.

10.9 The cluster

- Removal is a **cluster operation**: the change writes cluster configuration version N+1 and removes the module's placement; every node closes its gate on receipt (unplaced nodes only stop forwarding) and confirms; the coordinator reports *3/3 confirmed*. A placed node that does not confirm within `drishti.modules.confirm-seconds` (30) is left out of routing until it does.
- Every node's **module fingerprint** (section 4.11.6) changes; a node that starts later with the `.dmod` still on its disk shows **amber** (*present but not placed*) in the node × module matrix and serves nothing of it; a late joiner receives only what is placed on it.
- Consoles follow the cluster registry's configuration version (section 4.11.4) and drop the module's cached console bundle: a console's route table is never ahead of or behind the cluster by more than one event.

10.10 What people see: the removed-module page, 410 or 404

- **Removed** module's page, asset or API: **410 Gone** with `Cache-Control: no-store`. Pages render the kernel's notice page (in the kernel shell, so menus and the command line still work): *This page belonged to Genomics, which is no longer installed on this system.* with *Go to Home* and *Search*. **Administrators** also see *removed by alice, approved by bob on 2026-11-08 10:42 (CHG-2026-1108-0057)* and a link to Admin → Modules. APIs answer an RFC 7807 problem: `DRS-1028`, *Module not installed*, `module: genomics`.
- **Disabled** (or unlicensed, or refused) module: **404**, as today (*not available: the genomics module is off; ask your administrator*): the module is still installed and may come back; nothing permanent is claimed.
- **Never installed**: the ordinary 404 (no tombstone, so nothing reveals what other sites install).
- **Recommendation** (question Q12): **410 for removed, 404 for off**; tombstones are what make 410 possible and are kept 365 days.

10.11 The "absent everywhere" guarantee, and how it is tested

Guarantee: after a module is removed (or switched off), no person, page, API, search, Help, job, cache, AI tool or registry anywhere on any node can reach anything of it; what stays is the site's data, people's saved work (marked) and the audit.

The absence suite (`AbsenceSuite` in the testkit; the kernel runs it on the genomics sample and banking; every module runs it as conformance C14):

1. **Install** the module in the harness (kernel + module, demo lake, two people signed in, one composed page open in a headless browser, one saved view, one workspace tile, one scheduled export).
2. **Record every surface** into an inventory: the console route table and every URL it can produce; the sitemap per lens; the function list and type-ahead for each word; a search for each title, word and glossary term; the Help index; the asset list; the server's mapped endpoints (OpenAPI); the scheduler; the Python items; `ModuleContributions` entries; AI tools; cache reports; metrics names; the designer palette; settings pages; exports; event subscribers.
3. **Remove** it (the four-eyes path with a test approver).
4. **Assert each surface is gone**: every recorded URL 410 (404 for the *off* variant) with the right page or problem; every list, index and registry without the module's entries; the kernel's own entries unchanged (the inventory diff is exactly the module's).
5. **Assert the open tab** shows the notice, logged **no JavaScript error**, and made no further request to the module's paths.
6. **Assert saved work** is marked, not changed or deleted; the export paused; the two people still isolated.
7. **No references left**: a reachability scan from every kernel registry (contributions, caches, scheduler, event bus, AI tools, route table, Jinja environments) finds **no object whose class loader is the module's**, and no Python object from `drishti_modules.<id>` (in P5 also not in `sys.modules`).
8. **Class loader released** (P5 hot unload, and in the harness from P2): a `WeakReference` to the module's class loader is cleared after GC within a bounded number of cycles; repeated 50 times (install/remove) without growth in metaspace or threads named `mod-<id>-…`.
9. **Reinstall** and assert everything recorded is back and saved work resolves again.

A failure names the surface and the leftover (*search still finds "Variant explorer" (index entry 41922, tag module:genomics)*).

---

11. Security

11.1 Signing

- Every `.dmod` is signed (Ed25519 over the hash list). Drishti's **module publisher root** signs publisher certificates (Drishti itself for banking, economy, capital markets; partners later). The site trusts the root by default (`config/trust/publishers/drishti-root.pem`) and may add its own publishers (taint `U`).
- Keys: private keys in an HSM or an offline key file, never in a repository; `module sign` takes a PKCS#11 URI or a file.
- Revocation: a revocation list file shipped with each kernel release and loadable by the site; a revoked certificate refuses its modules.
- The kernel jar and connector plugins are signed the same way (P4).

11.2 The licence

The signed licence of ACCESS_MODEL.md section 7 (built in i, after the security discussion) names **features**; each module's `licence.feature` must be in it (V5). Limits (`positions`, `nodes`, `users`) are read by the registry and offered to the module through `ModuleContext.licence()` (read-only, the module cannot change them). The licence check is the kernel's; modules know nothing of it beyond the limits.

11.3 Sandboxing

Part	Isolation
Java	own class loader, sees only the kernel API; no `System.exit`, no new threads outside the kernel's pools (an SDK architecture test; the kernel's thread factory names threads `mod-<id>-…` for attribution); file access only to its own data folder through `ModuleContext.folders()` (checked by the conformance suite; the JVM has no Security Manager any more, so this is a review and test rule, not a runtime wall: Q5)
Python	the kernel's strict sandbox (no network interface, Landlock, audit hook), per-run import path, the person's row rules on every frame
Console Python	runs in the console process; restricted to its plugin hooks; the static import guard (no core imports of modules, no module imports of other modules)
JavaScript and CSS	served from `/m/<id>/static/<version>/`, under the console's CSP (`script-src 'self'` plus SRI hashes from the module's manifest; no inline script, no `eval`, no remote origins)

11.4 Secrets

Never in a module (5.10). A module has no secret API. Its own sensitive settings (none for banking) would be references the site fills (`${secret:name}`) and only the kernel resolves.

11.5 Session isolation

A module runs in a multi-user, concurrent server: the module conformance suite runs two people at once and fails on any per-person state in statics, singletons or shared caches without the person's identity (the session-isolation rule; cross-person leaks are P1 and release blockers). `Kinds` reads are always **as the person**.

11.6 Audit

Install, upgrade, rollback, remove, switch, binding change, refusal, taint, and every run of a module's API and Python items carry the module id and version in the audit line.

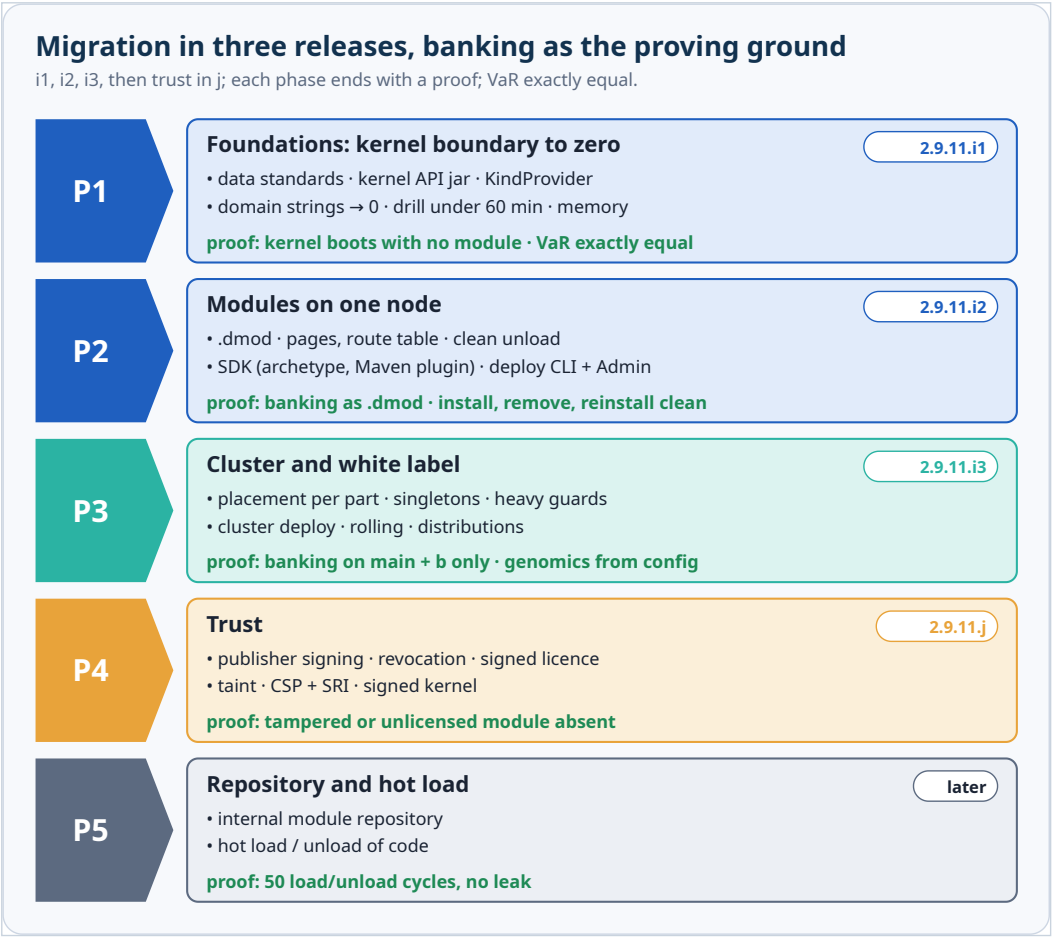
---

12. Migration, with banking as the proving ground

12.1 Where we are (surveyed on `89f010df5`)

Already modular (built in the e+f+g train)	Still leaking or not done
Java split: `drishti-banking`, `drishti-compute-banking`; cycles J1–J6 broken through SPIs; `ArchitectureRulesTest`	**39 kernel files with 77 banking strings** on the rule-4 allow list (`tools/domain_strings_allowed.yaml`): bicompute (Explain, functions, what-if, scale), the console's `core/edition.py` (14), `core/facts.py`, `drishti_pivot.js`, `EngineCatalogue`, `ModuleManifest`, `EmbeddedAccessRegistry`, `ErrorCode`
Manifests (`META-INF/drishti/module.yaml`, `lenses.yaml`, `engines.yaml`, `token-scopes.yaml`) in the jar; the registry decides	**The thin jar is not built**: banking is always on the class path; "installed" = in the fat jar
`@ConditionalOnModule`, module switch, `DRS-1025` / `1026`; module off = gone everywhere (tests)	**Console plugins live inside the repo's console** (`drishti-console/plugins/{banking,economy}`), imported as `plugins.<id>`
Console plugin hooks C1–C7; static assets by module and lens; Elements loader refuses a module's library	**Modules depend on `drishti-server` internals** (no kernel API jar yet)
Packs, lakes, BI, tables, reports, lenses in module folders (`config/*/banking/`)	**Banking ships connector plugins** (`var-positions`, `var-pv-stream`) and reactive connectors; contracts name connectors (`storage.connector`)
Editions folded into lenses: done (`config/lenses/<module>/`)	**No signing, no module path, no distribution file**; branding split across `ui.*`, `drishti.branding.*` and lens titles
Packs name their domain; Help by module (`plugins/banking/help.yaml`)	Banking Python CLIs (`varcli.py`, …) and the guides/tutorials live in the repo's shared folders

12.2 Phases



Phase	Deliverables	Proof (gate)
P1 · Kernel boundary to zero (**i1**, with data standards and the drill under 60 minutes)	`drishti-kernel-api` jar (section 2.3: existing SPIs moved under `kernel.api`, new `DrishtiModule`, `ModuleContext`, `KindProvider`, `ModuleSettings`, `AiToolProvider`, `@ModuleApi`); banking compiled against it only; the 39-file allow list to **0**; `KindProvider` replaces banking's connector plugins; contracts' `storage:` → `binding-hint:`; `config/bindings.yaml` + `bindings migrate`; kernel API 1.0 frozen with `japicmp` baseline	kernel boots with **no modules** (economy and capital markets off too) and serves BI, Pivot, Build, Help, GLEIF; `domain_strings` empty; banking passes its suite; **VaR exactly equal**
P2 · Modules on one node (**i2**)	the `.dmod` format and packager (`module build`, unsigned dev key), `modules.path`, discovery and V1–V4, V6–V7 checks, class loader per module + child Spring context, REST mounting, Python per-module import path, console plugins loaded from module folders (`drishti_modules.<id>`, `/m/<id>/static/`), **plugin.yaml format 2, the live route table, per-module template environments, the API proxy** (section 9), **the unload sequence, tombstones and the absence suite** (section 10), Help merge, **the thin jar**, `module list | info | install | remove | verify`, **single-node deployment** (`module deploy` plan/apply and Admin → Modules → Deploy over the deployment API, four eyes: section 7.6), Admin → Modules (mock-up a), Admin → Data bindings (mock-up b), launchers/Docker/ECS/stack with the modules folder; the SDK (parent pom, archetype, `drishti-module-maven-plugin`, harness, `module new` / `run`); banking, economy and capital markets as `.dmod`s built as Maven projects; the developer guide + tutorial	**banking passes against the kernel alone** (the harness: kernel thin jar + `banking.dmod`, nothing else); **genomics and banking pass the absence suite**; the full drill on the thin jar + three modules; a drill with **no modules**; VaR exactly equal
P3 · Cluster and white label (**i3**)	**placement** per node or role and per part, singletons, the heavy-work guards, `config/placement.yaml`; **cluster deployment** (plan and apply across the placed nodes, two-phase activation, late joiners, the node × module matrix, rolling upgrades pinned per session: section 4.11); the distribution file and `dist build`; branding literals to 0 (the 26-literal fix and its guard); `drishti.finance` (banking + capital markets + economy) as the default product; the **`genomics-sample`** module and `drishti.genomics`; white-label mock-up (c) built; Help/docs by distribution	**`drishti.genomics` boots from configuration only** (same kernel build, a different file and module set): no banking word on any page, in search, Help or assets (a crawler test); `drishti.finance` equals today's product (screenshots); **banking placed on main and b only, nothing on c**; failover stays inside the placement
P4 · Trust (**j**, the security release)	signing with publisher keys, trust store, revocation, V5 with the signed licence, taint, `ModuleMigration`, CSP with SRI for module assets, signed kernel and connector plugins	tampered `.dmod` refused; unlicensed module absent; a revoked publisher's modules absent; rollback restores VaR exactly
P5 · Repository and hot load (later)	an internal module repository (search, fetch, dependency download), hot code load/unload with the leak test, module marketplace only if the owner wants one	50 load/unload cycles without a class-loader leak; hot upgrade under load with no failed request

Three releases (decided by the owner, 2026-11-08; Q8): 2.9.11.i1 · Foundations (data standards: Iceberg onto iceberg-core, Python Delta reads through Delta Kernel, a critical review of the other connectors and sources; P1; the full drill under 60 minutes; memory and speed; about the size of g), **2.9.11.i2 · Modules on one node** (P2; about 1.2 × g), **2.9.11.i3 · Cluster and white label** (P3; about the size of g). P4 is in **2.9.11.j** (trust and

security); then k (risk engines II), l (collateral), m (CVA), n (hardening); new domains after the line. P5 later. The rows are in IMPLEMENTATION_PLAN.md.

12.3 Banking step by step (P1–P2)

1. Survey again; freeze the boundary list (refresh DOMAIN_MODULES.md section 2).
2. Create `drishti-kernel-api`; move the SPI interfaces (package rename `com.ash.drishti.kernel.api.*`; the old types deleted, not aliased: banking is the only implementer and moves in the same branch).
3. Banking's `pom.xml` depends on `drishti-kernel-api` (`provided`) and drops `drishti-server`; fix each compile error by a kernel API service or a new SPI; the architecture test *module imports only the kernel API* turns on.
4. Empty the allow list file by file: words to the module's manifest, engine knowledge to the engine catalogue, `edition.py`'s banking words to lens files, `drishti_pivot.js`'s MR bits to the plugin's `screens.pivot`.
5. `var-positions`, `var-pv-stream` → `KindProvider`; `var-engine`, `exposure-engine` reactive connectors → computed kinds; contracts' `storage:` → `binding-hint:`; `bindings migrate` on the demo and test sites.
6. The packager: lay banking's files into the Maven project of section 8.10 (project-mapping.txt) and the `.dmod` layout (a `git mv` of the plugin, packs, lenses, BI, lakes, docs into a `modules/banking/` source tree in the repo; the repo remains one repository, the module builds separately from it).
7. The thin jar; launchers gain the modules folder; the drill installs `banking.dmod`.
8. The VaR check, both drills.

12.4 Risks and mitigations

Risk	Mitigation
VaR drifts during the moves	the exact-equality VaR check on every banking branch's pre-merge gate, not only at the drill
Spring child contexts break auto-configuration ordering or AOP across loaders	prototype in week 1 on `economy` (content-only) then on banking's smallest engine (SIMM); keep the shared library list small; a test per SPI crossing the boundary
Class-loader leaks (threads, caches, ThreadLocals)	kernel-owned thread pools only; the leak test in the harness from P2 even though hot unload is P5
Size of i (2–3× g)	**split into i1, i2, i3** (decided, Q8): each with its own proof and drill
Packaging change across every launcher, Docker, ECS, stack and the drill (why the thin jar was deferred)	one branch for all launchers with a launch-matrix test; the drill's first run on the thin jar is a dry run without timing
Two ways during migration (in-jar and `.dmod`)	none: the branch that introduces the module path removes the in-jar path in the same merge; tests and harness use the `.dmod`
Python code paths across modules	per-run import path; a test that one module's run cannot import another's package
Console import renames (`plugins.banking` → `drishti_modules.banking`) break bookmarks or extensions	addresses kept via `paths:`; only Python import paths change (internal)
Performance (an extra class-loader hop, per-module DispatcherServlet)	measured in the benchmark: VaR request latency and pivot throughput before/after, budget ≤ 2%
Windows QA (no .exe)	the packager and loader are Java + Python only; `.dmod` is a zip; signature checks in Java; tests run on Windows
Docs churn (guides move into modules)	guides keep their slugs; Help merges them; the docs gates read module docs

13. Open questions for the owner

#	Question	Recommendation
Q1	The core pack's finance entities (`book`, `desk`, `trader`, `counterparty`, `legal-entity`, ...): kernel or banking? (was D1)	move them to a **`banking-reference` pack in the banking module**; the kernel keeps `calendar` and generic entities (organisation, person, location); GLEIF stays kernel
Q2	Bindings file vs connectors' `kinds:` lists	**`config/bindings.yaml` is the one home**; `kinds:` read for one release, then removed
Q3	Module static files at `/m/<id>/static/<version>/` (new addresses) or today's `/static/…`	**new addresses** (versioned cache, per-module CSP, clean unload); page addresses unchanged
Q4	Per-module bindings of one kind (`for:`)	**not in i**: one binding per kind; per-person views are row rules; revisit with a real case
Q5	Java isolation without a Security Manager: accept review + tests, or run untrusted modules out of process	**in process for Drishti-signed and partner modules**; out-of-process modules (a gRPC/Flight boundary) only if a third-party marketplace is wanted (P5+)
Q6	The market-data pack and feed: banking or capital markets	**capital markets owns market data** (pack and feed); banking `requires: capitalmarkets` (optional today, required once its engines read it); closes DOMAIN_MODULES D6's interim
Q7	Who signs: one Drishti key for everything, or publisher certificates under a Drishti root	**a root + publisher certificates** (Drishti, then partners), revocation list per release
Q8	Split i	**DECIDED (owner, 2026-11-08): three releases**: i1 foundations (data standards, P1, drill under 60 minutes, memory and speed); i2 modules on one node (P2); i3 cluster and white label (P3); P4 in j
Q9	Repository layout: banking's sources stay in this repository under `modules/banking/` or move to their own repository	**stay in this repository** (one drill, one history) but build separately into a `.dmod`; a separate repository when a partner module exists
Q10	Default distribution of the repo's build	**`drishti.finance`** (today's product); `drishti.genomics` as a sample only, never shipped to customers
Q11	The kernel API version's start	**1.0 at the end of P2** (P1's API is 0.x and may change freely)
Q12	HTTP status for a removed module's pages and APIs: 410 Gone or 404	**410 for removed** (tombstones), **404 for off, refused or unlicensed** (it may come back; nothing permanent claimed), the plain 404 for never installed; all `no-store` (section 10.10)
Q13	Hot page install: P2 or P3	**P2 for off, on and remove** (the guarantee needs it) **and for declarative console parts** (Sutras, templates, static, menus); console Python handlers and server code at the rolling restart until P5 (section 9.8)
Q14	Console Python handlers in partner modules, or templates and Sutras only	**allowed, but optional and small**: context builders only (no routes, no request, no state), lint + review; a partner module with no handlers installs fully hot. Revisit if a marketplace comes (Q5)

#	Question	Recommendation
Q15	Top-level path aliases (`/var`) for partners, or only Drishti-signed modules	**Drishti-signed only by default**; a site may grant one partner module aliases (`allow-aliases`, four eyes); command words for all, checked against the reserved names (section 9.9)
Q16	How long tombstones (and the *needs module X* marks) are kept	**365 days** (`drishti.modules.tombstone-days`), then the artefacts show *needs a module that is no longer known* and are still never deleted automatically
Q17	Where the SDK and modules are published	**an internal Maven repository** (the one the build already uses) for the SDK artefacts and `.dmod` files from P2; the P5 module repository adds search and dependency download on top of it
Q18	Plugin format 1 (`routers:`) after P2	**read for one release with a deprecation line in Health, then refused**: banking, economy and capital markets move to format 2 in the P2 branch itself (no dual machinery)
Q19	Placement by role or by node name as the recommended default (section 4.11.1)	**by role** (`role:risk-compute`): a new node with the role is staged automatically, replacing a machine changes no placement, and roles already drive Drishti*net access; **node names allowed** for one-off placements (a single GPU box) and shown as such in Admin
Q20	Deployment from scripts: the CLI only, or also a standalone `tools/modules/deploy_module.py`	**the CLI (`drishti.py module deploy`) plus thin `tools/deploy-module.sh` / `.ps1` wrappers** for schedulers and CI (the project's script convention); no second Python implementation, both front ends call the one deployment API

14. What moves from DOMAIN_MODULES.md

DOMAIN_MODULES.md remains the record of the split as built (sections 1–5, 8) until P2 completes, then is deleted (git keeps it) and its rules move into the developer guide. From today:

DOMAIN_MODULES.md	Moves to
section 6 step 9 (the thin jar and the plugins folder) and section 8 *Phase 2, still to do*	this document, section 12 (P2)
section 7 D1 (core pack's finance entities), D6's interim (market-data pack)	section 13, Q1 and Q6
section 4 rule 4 (strings) with its allow list	section 1.3 principle 1 and P1's gate (empty)
section 3 (the layout as built)	stays until P2, then replaced by section 3.2 here

ACCESS_MODEL.md stays the home of the access chain and the registry; this document adds the package, the loader and binding, and only links back.

15. Mock-ups

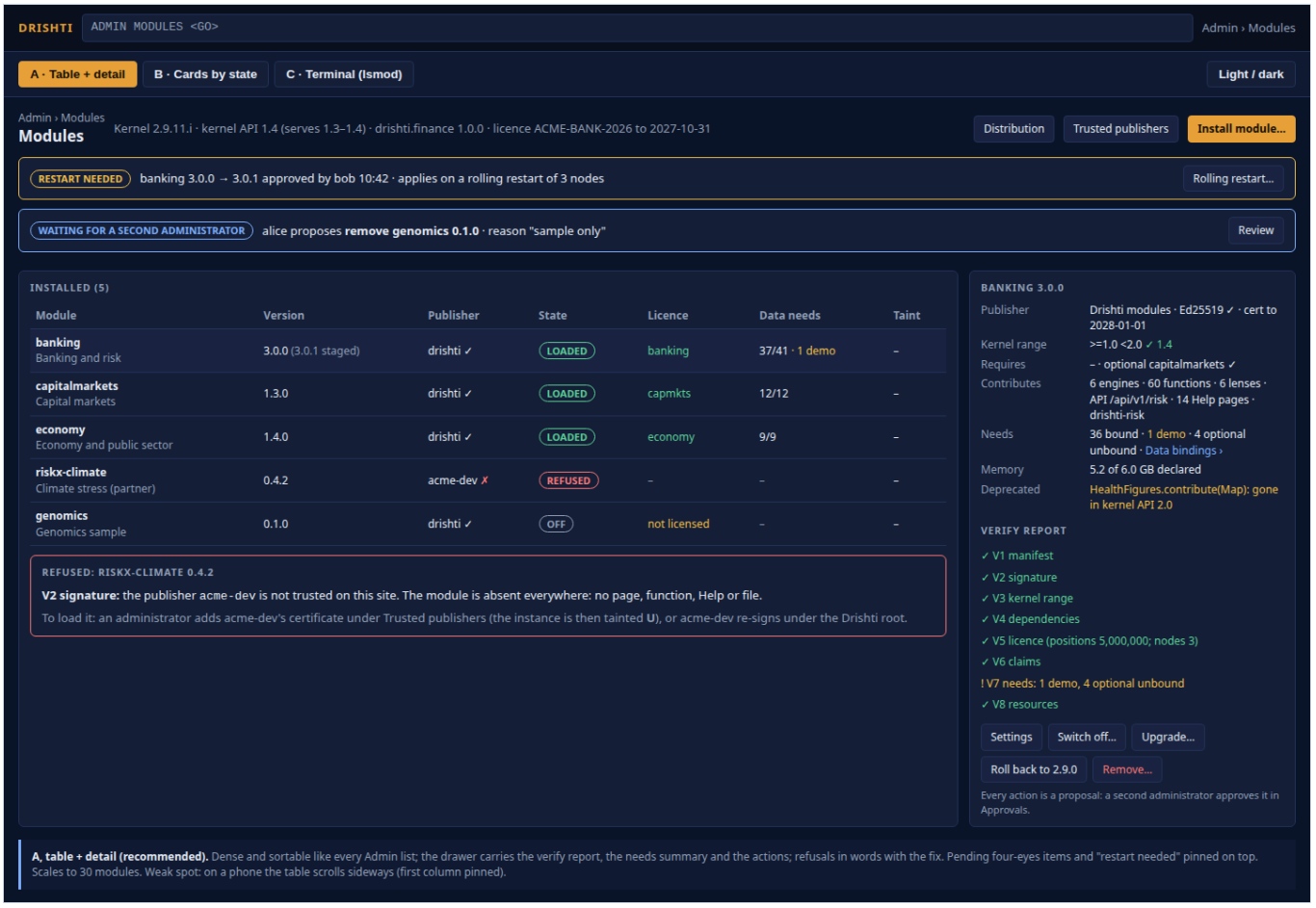
Static, self-contained (no CDN), in Drishti's terminal look with light and dark; each file holds the alternatives as views of one page (A the recommended default, per the A/B/C rule), with the trade-offs written under each.

Screen	File	Alternatives	Recommendation
(a) Admin → Modules	mockups/admin-modules.html	A table + detail drawer · B cards by state · C terminal `lsmod`	**A** (dense, sortable, the drawer carries the verify report and four eyes); C as the terminal layout
(b) Admin → Data bindings	mockups/data-bindings.html	A needs matrix (kind × module, source per row) · B by module (needs list per module) · C data map (sources → kinds → modules)	**A** (one row per kind shows sharing and gaps at once); B as the module's own tab; C later as a read-only picture
(c) White label: Finance vs Genomics	mockups/white-label.html	A side by side · B one console with a distribution switch · C the distribution file beside its result	**A** for the owner's review; C is the authoring view in Admin → Distribution (P3)
(d) Module removed: an open page and a saved view	mockups/module-removed.html	A in place: the page frozen under a notice bar, saved views kept in their list with a *needs genomics* chip · B replaced: the page swapped for a full notice card, saved views moved to an *Unavailable* group	**A** (people keep their bearings and their work stays where they left it; nothing jumps); B for phones
(e) Admin → Modules → Deploy	mockups/module-deploy.html	A step wizard (choose, place, plan, approve, apply, result) · B one page, the plan matrix updating live as placement changes · C split view: placement on the left, the plan on the right	**C** (placement and its consequence side by side, the plan always visible before approval; it steps through approval and apply in the right pane); A for first-time administrators as a *guided* toggle; B gets crowded with more than five nodes

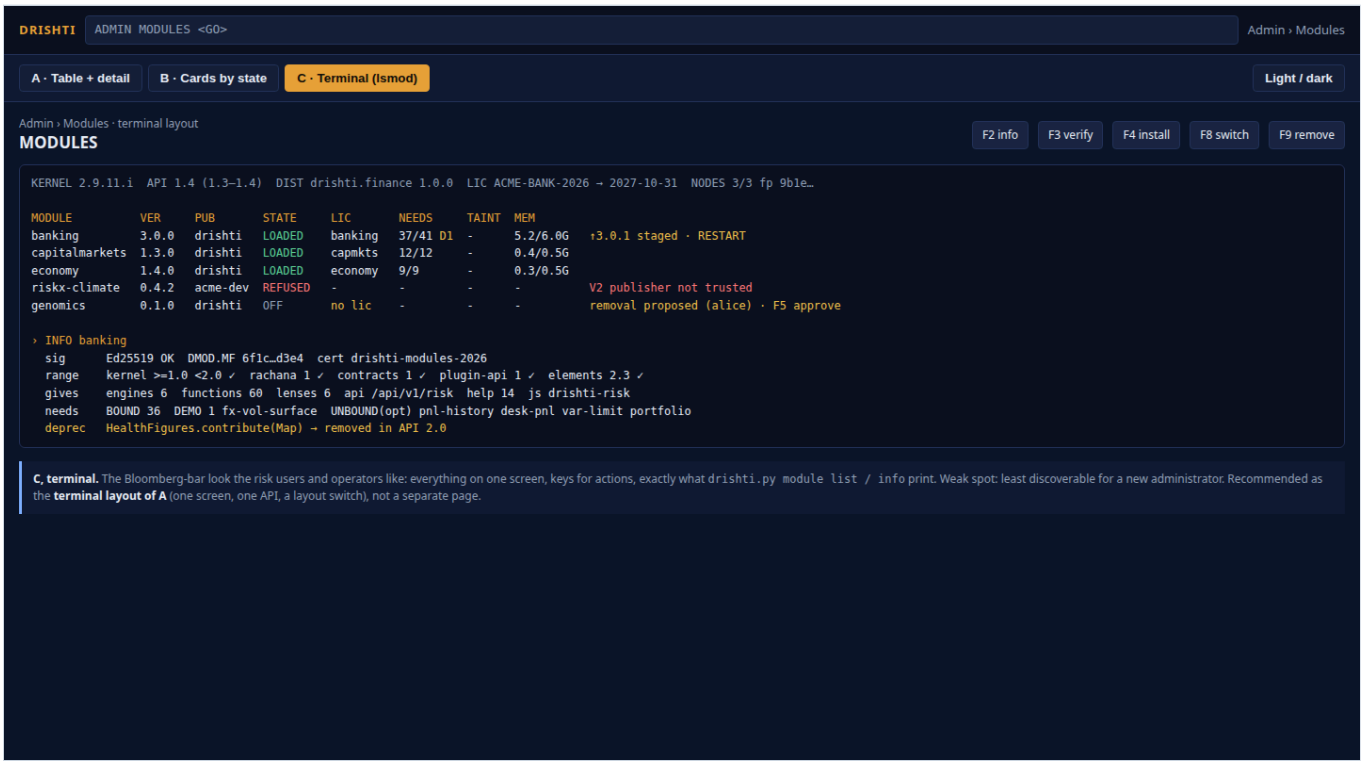
15.1 The recommended options

Screenshots of the static pages (dark theme; each page also has light). The other options are in the same pages (buttons A, B, C) and as pictures beside them (`admin-modules-B.png` …).

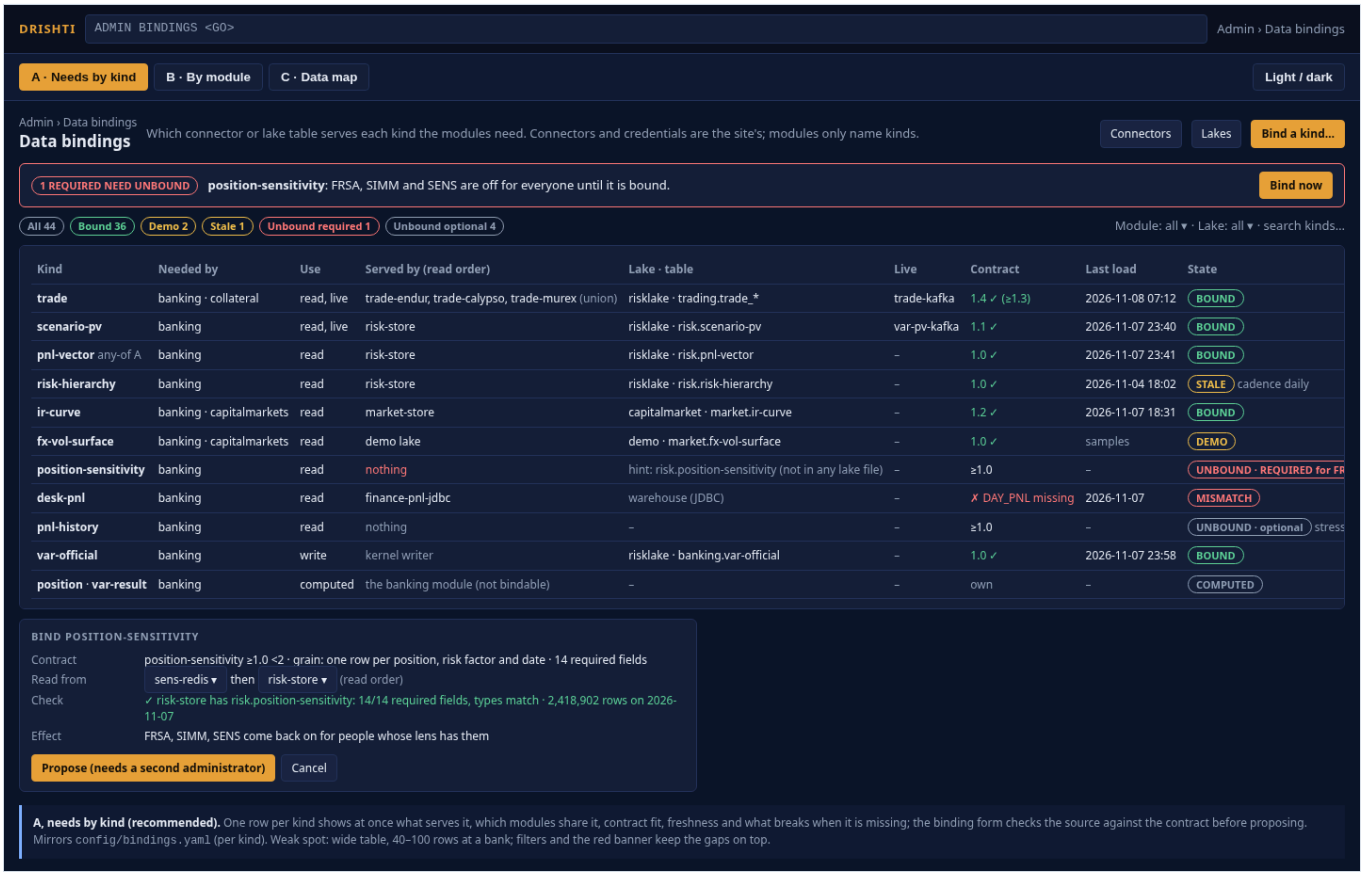
(a) Admin → Modules, option A (table + detail drawer, the verify report, four-eyes items and *restart needed* on top):



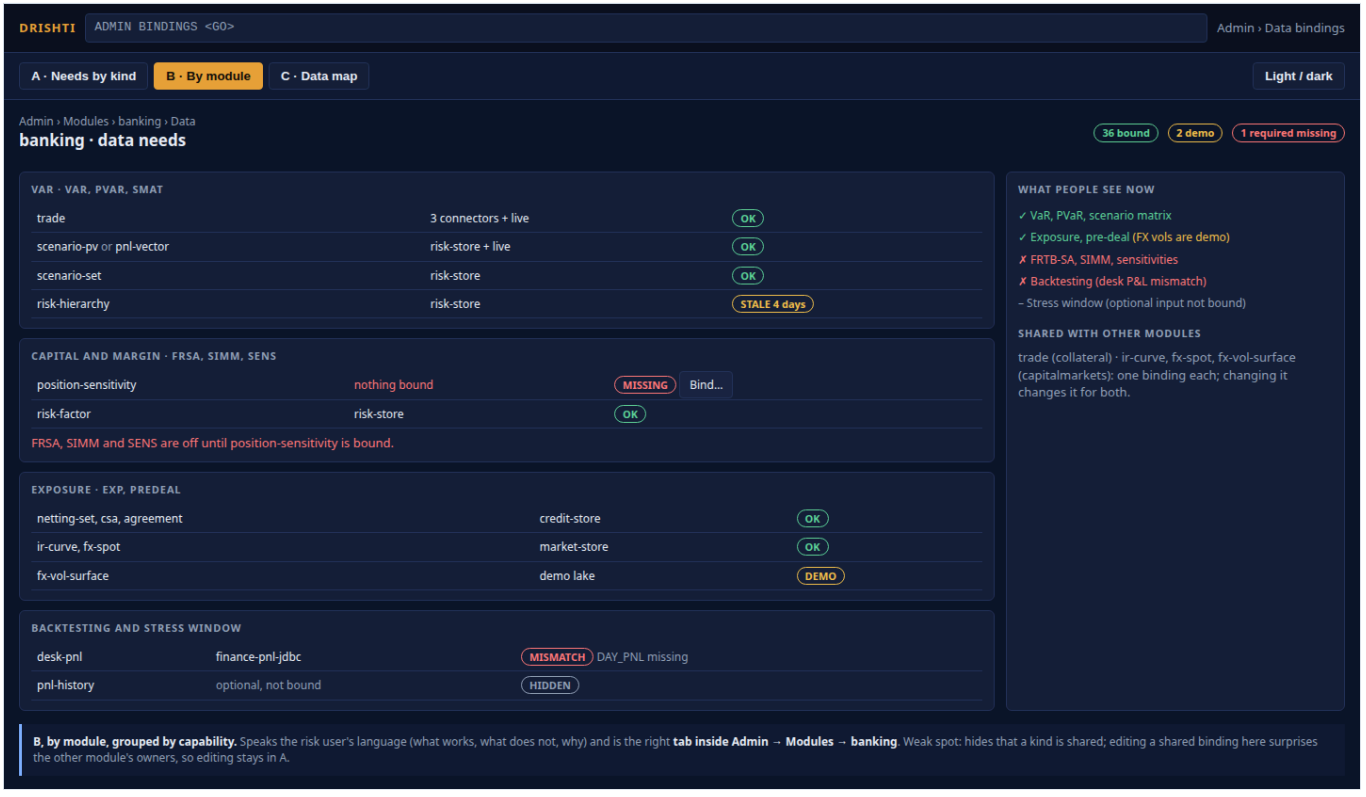
(a) Admin → Modules, option C (the terminal layout of the same page):



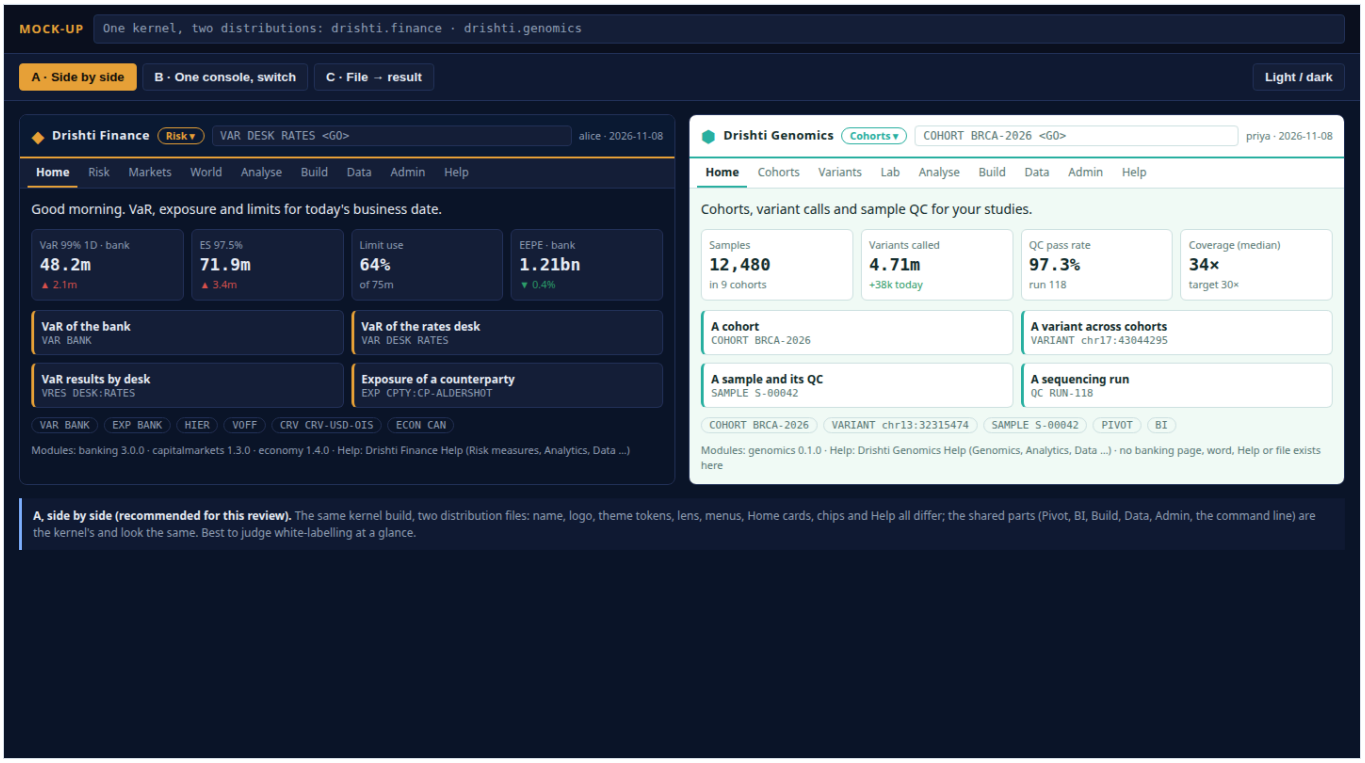
(b) Admin → Data bindings, option A (needs by kind, sharing, contract fit, freshness; the bind form checks before proposing):



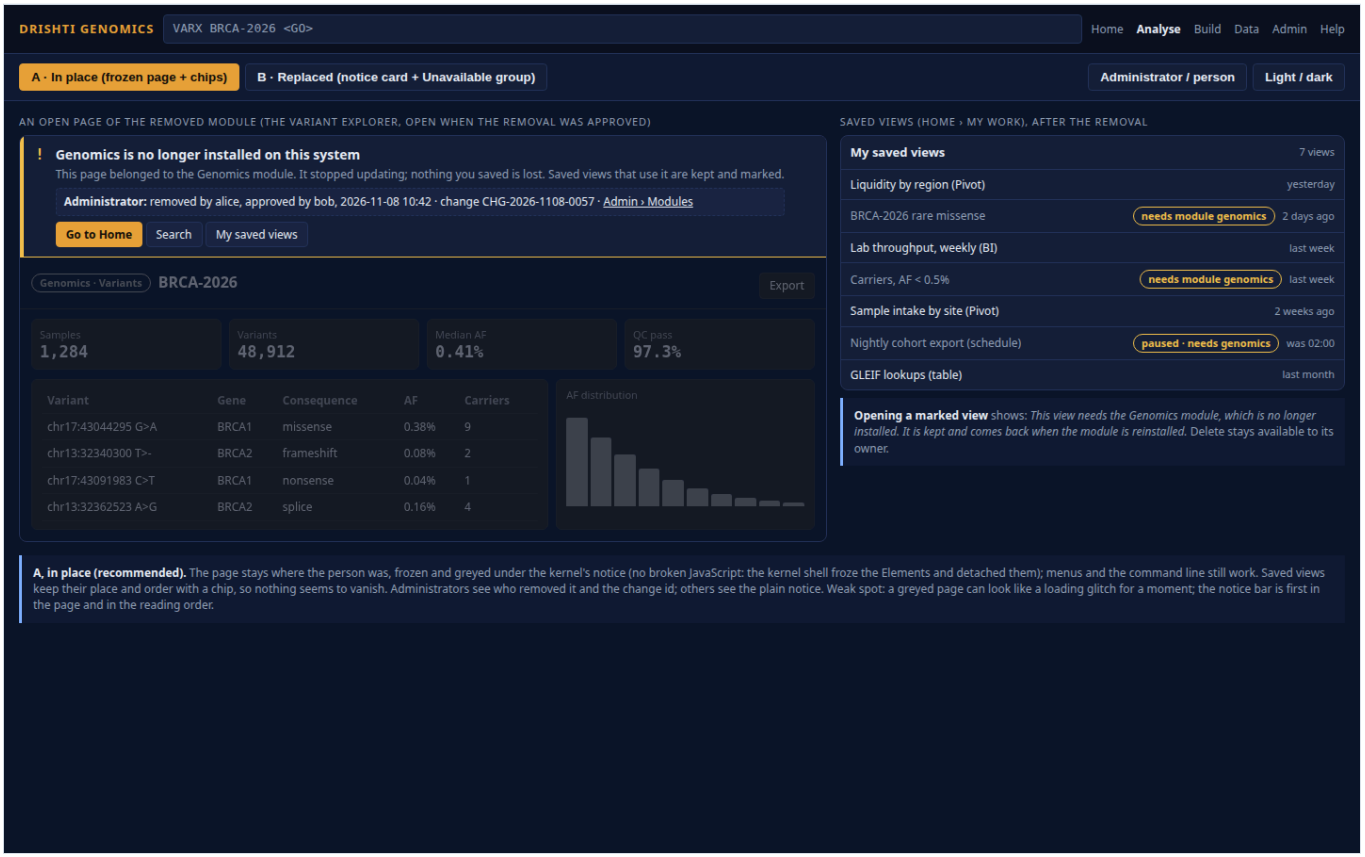
(b) option B (by module, the tab inside Admin → Modules → banking):



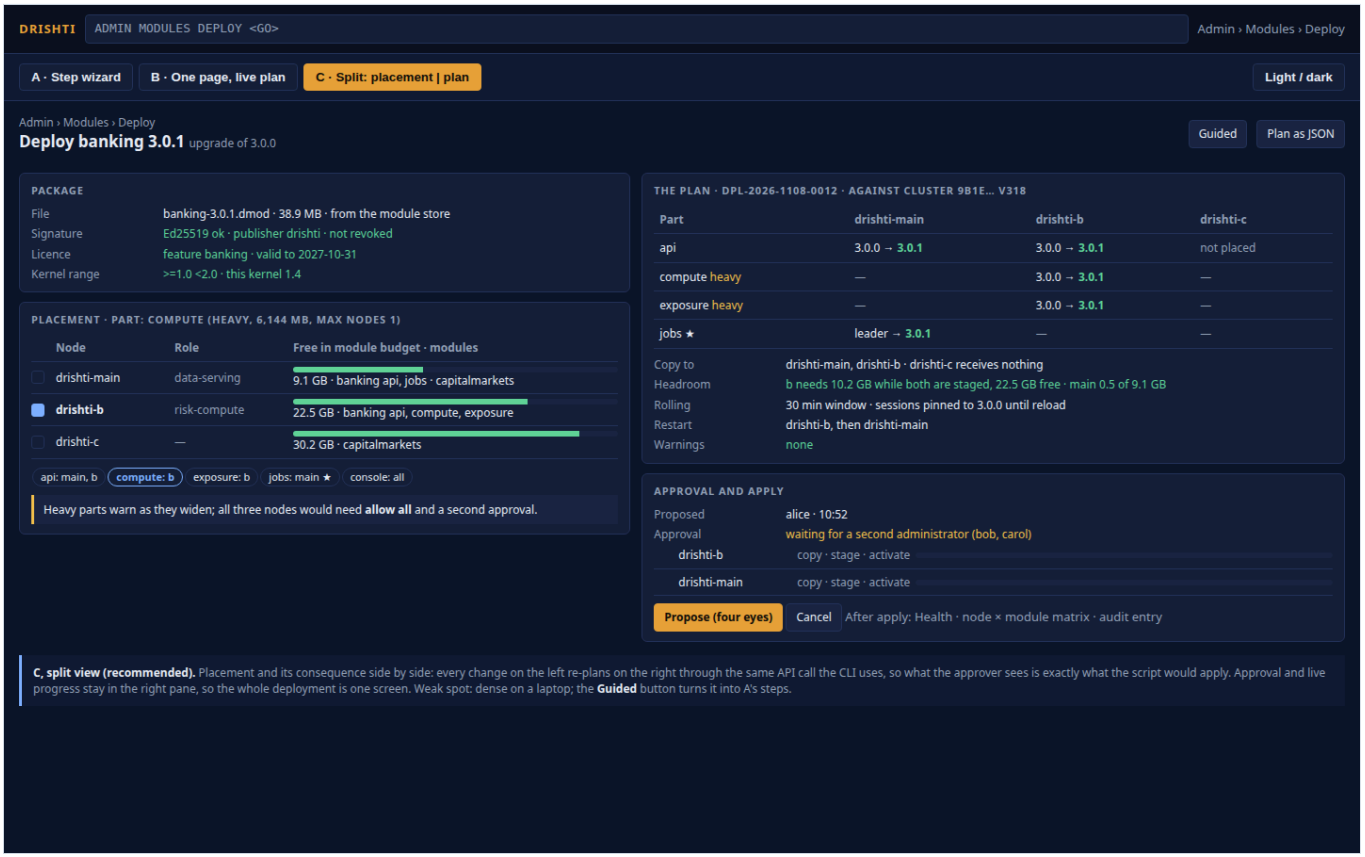
(c) White label, option A (Drishti Finance and Drishti Genomics side by side, one kernel):



(d) Module removed, option A (the open variant explorer frozen under the kernel's notice, the administrator's reason line; saved views with the *needs module genomics* chip, kept in place):



(e) Admin → Modules → Deploy, option C (placement per part on the left with the node picker; the plan matrix, headroom and warnings on the right; then approval and live apply in the same pane):


DRISHTI
ADMIN MODULES DEPLOY <GO>
Admin › Modules › Deploy
A · Step wizard
B · One page, live plan
C · Split: placement | plan
Light / dark
Admin › Modules › Deploy
Deploy banking 3.0.1 upgrade of 3.0.0
Guided
Plan as JSON
PACKAGE
File banking-3.0.1.dmod · 38.9 MB · from the module store
Signature Ed25519 ok · publisher drishti · not revoked
Licence feature banking · valid to 2027-10-31
Kernel range >=1.0 <2.0 · this kernel 1.4
PLACEMENT · PART: COMPUTE (HEAVY, 6,144 MB, MAX NODES 1)
Node Role Free in module budget · modules
drishti-main data-serving 9.1 GB · banking api, jobs · capitalmarkets
drishti-b risk-compute 22.5 GB · banking api, compute, exposure
drishti-c — 30.2 GB · capitalmarkets
api: main, b
compute: b
exposure: b
jobs: main ★
console: all
Heavy parts warn as they widen; all three nodes would need allow all and a second approval.
THE PLAN · DPL-2026-1108-0012 · AGAINST CLUSTER 9B1E… V318
Part drishti-main drishti-b drishti-c
api 3.0.0 → 3.0.1 3.0.0 → 3.0.1 not placed
compute heavy — 3.0.0 → 3.0.1 —
exposure heavy — 3.0.0 → 3.0.1 —
jobs ★ leader → 3.0.1 — —
Copy to drishti-main, drishti-b · drishti-c receives nothing
Headroom b needs 10.2 GB while both are staged, 22.5 GB free · main 0.5 of 9.1 GB
Rolling 30 min window · sessions pinned to 3.0.0 until reload
Restart drishti-b, then drishti-main
Warnings none
APPROVAL AND APPLY
Proposed alice · 10:52
Approval waiting for a second administrator (bob, carol)
drishti-b copy · stage · activate
drishti-main copy · stage · activate
Propose (four eyes)
Cancel
After apply: Health · node × module matrix · audit entry
C, split view (recommended). Placement and its consequence side by side: every change on the left re-plans on the right through the same API call the CLI uses, so what the approver sees is exactly what the script would apply. Approval and live progress stay in the right pane, so the whole deployment is one screen. Weak spot: dense on a laptop; the Guided button turns it into A's steps.